

# آموزش AVR

تهیه و تألیف: محمد مهدی قائمی

[mahdighaemi123@gmail.com](mailto:mahdighaemi123@gmail.com)

codewithmahdi.ir

@codewithmahdi

v1.0.0

## فهرست مطالب

|                                         |    |
|-----------------------------------------|----|
| فصل ۱: مفاهیم پایه و آماده‌سازی ابزارها | ۵  |
| ۱ تفاوت میکروکنترلر و پردازنده معمولی   | ۵  |
| ۲ محیط توسعه نرم‌افزار یا IDE           | ۶  |
| ۳ محیط شبیه‌سازی پروتئوس                | ۷  |
| ۴ ساختار کلی یک برنامه AVR              | ۷  |
| فصل ۲: کالبدشکافی سخت‌افزار ATmega32    | ۹  |
| ۱ بررسی پایه‌ها و ساختار فیزیکی         | ۹  |
| ۲ سیستم کلاک و نوسان‌ساز                | ۱۰ |
| ۳ پایه ریست یا شروع مجدد                | ۱۱ |
| ۴ انواع حافظه در AVR                    | ۱۱ |

### فصل ۳: جادوی رجیسترها ۱۳

- ۱ مفهوم رجیستر و نقش آن..... ۱۳
- ۲ سه تفنگدار پورت‌ها DDR و PORT و PIN..... ۱۳
- ۳ منطق باینری و بیت‌وايز..... ۱۴
- ۴ تعیین جهت پورت‌ها با کدنویسی..... ۱۵

### فصل ۴: اولین پروژه عملی؛ چشمک زدن LED ۱۷

- ۱: طراحی سخت‌افزار در پروتئوس..... ۱۷
- ۲: چالش زمان و تابع تأخیر..... ۱۸
- ۳: کدنویسی در Atmel Studio..... ۱۹
- ۴: تحلیل خط به خط برنامه..... ۲۰
- ۵: تست نهایی..... ۲۰

### فصل ۵: خواندن ورودی‌ها؛ تعامل با کلیدها ۲۱

- مفهوم ورودی و مشکل نویز..... ۲۱
- ۱: بستن مدار در پروتئوس..... ۲۲
- ۲: کدنویسی و فعال‌سازی مقاومت داخلی..... ۲۲
- ۳: تحلیل منطق شرط IF..... ۲۴

### فصل ۶: بازی با اعداد؛ راه اندازی سون سگمنت (7-Segment) ۲۵

- کالبدشکافی سون سگمنت..... ۲۶
- نگاشت بیت‌ها؛ تبدیل نقاشی به صفر و یک..... ۲۶
- چرا کد باینری (دودویی) بهتر از هگزاسیمال است؟..... ۲۷
- مرحله ۱: بستن مدار در پروتئوس..... ۲۸
- مرحله ۲: کدنویسی شمارنده با استفاده از آرایه..... ۲۸
- تحلیل نهایی و عمیق برنامه..... ۳۰

## فصل ۷: حرف زدن با کاربر؛ راه اندازی نمایشگر LCD ۳۲

- ۳۲..... زبان ال سی دی؛ پایه RS چیست؟
- ۳۳..... مرحله ۱: بستن مدار در پروتئوس
- ۳۴..... مرحله ۲: کدنویسی درایور LCD
- ۳۶..... تحلیل نهایی

## فصل ۸: پل میان دیجیتال و آنالوگ (ADC)؛ ساخت دماسنج ۳۷

- ۳۸..... مفهوم ADC: تبدیل ولتاژ به عدد
- ۳۸..... آشنایی با سنسور LM35
- ۳۸..... مرحله ۱: بستن مدار در پروتئوس
- ۳۹..... مرحله ۲: رجیسترهای ADC (اتاق فرمان)
- ۴۲..... تحلیل نهایی و عمیق برنامه

## فصل ۹: توقف ممنوع! جادوی وقفه‌ها (Interrupts) ۴۴

- ۴۴..... سناریوی پروژه
- ۴۵..... سخت افزار: پایه‌های خاص
- ۴۶..... کدنویسی پروژه وقفه
- ۴۷..... تحلیل و بررسی دقیق

## فصل ۱۰: ارباب زمان: تایمرها و کانترها (Timers) ۴۹

- ۴۹..... سناریوی پروژه: مولتی تسکینگ واقعی
- ۵۰..... مفهوم Prescaler تقسیم کننده فرکانس
- ۵۱..... کدنویسی: رقص هماهنگ تایمرها
- ۵۴..... تحلیل نهایی

## فصل ۱۱: پروژه نهایی؛ ساخت منوی هوشمند با کلید و LCD ۵۶

- ۵۶..... سخت افزار پروژه (مدار)
- ۵۷..... کدنویسی پروژه (سیستم منو)

## فصل ۱۲: دمیدن روح در کالبد؛ پروگرم کردن و نکات اجرایی

۶۶

بخش اول: معرفی ابزار انتقال (پروگرامر)..... ۶۶

بخش دوم: روش‌های اتصال سخت‌افزاری..... ۶۷

بخش سوم: فیوزبیت‌ها (Fuse Bits) ؛ بایوس میکروکنترلر..... ۶۸

بخش چهارم: مراحل گام‌به‌گام اجرا..... ۶۸

بخش پنجم: فوت کوزه‌گری؛ نکات اجرایی مهم..... ۶۹

# فصل ۱: مفاهیم پایه و آماده‌سازی ابزارها

به دنیای جذاب و قدرتمند میکروکنترلرها خوش آمدید. در این مسیر آموزشی قرار است پلی بزنیم میان دنیای نرم‌افزار و دنیای واقعی سخت‌افزار. هدف ما این است که یاد بگیریم چگونه با نوشتن کدهای زبان C، قطعات الکترونیکی را کنترل کنیم و سیستم‌های هوشمند بسازیم. در فصل نخست به بررسی مفاهیم بنیادین می‌پردازیم و محیط کار خود را آماده می‌کنیم.

## ۱ تفاوت میکروکنترلر و پردازنده معمولی

اولین سوالی که مطرح می‌شود این است که ابزار کار ما دقیقاً چیست. در کامپیوترهای شخصی قطعه‌ای به نام CPU وجود دارد که وظیفه پردازش را بر عهده دارد اما این قطعه به تنهایی ناتوان است و برای کار کردن به حافظه رم، هارد دیسک و مادربرد نیاز دارد. میکروکنترلر اما داستانی متفاوت دارد. میکروکنترلر یک تراشه کوچک است که تمام اجزای یک کامپیوتر را در دل خود جای داده است. یعنی پردازنده، حافظه موقت، حافظه دائم و درگاه‌های ارتباطی همگی درون یک قطعه سیلیکونی قرار دارند. این ویژگی باعث می‌شود بتوانیم سیستم‌هایی کوچک، ارزان و مستقل بسازیم.

چرا خانواده AVR را انتخاب کرده‌ایم؟ میکروکنترلرهای AVR ساخت شرکت Atmel از محبوب‌ترین گزینه‌ها برای شروع و حتی انجام پروژه‌های صنعتی هستند. این خانواده چند ویژگی کلیدی دارد:

- **سرعت بالا:** معماری این تراشه‌ها RISC است که باعث می‌شود دستورات با سرعت بسیار بالایی اجرا شوند.
- **پشتیبانی عالی از C:** ساختار داخلی AVR به گونه‌ای طراحی شده که سازگاری فوق‌العاده‌ای با زبان C دارد.
- **منابع فراوان:** مدل‌هایی مانند ATmega32 امکانات جانبی کاملی مانند مبدل آنالوگ و تایمرها را در اختیار ما می‌گذارند.

## ۲ محیط توسعه نرم‌افزار یا IDE

برای نوشتن برنامه نیاز به محیطی داریم که کدهای زبان C ما را دریافت کند و آن را به زبان ماشین تبدیل نماید. میکروکنترلرها تنها اعداد صفر و یک را درک می‌کنند بنابراین ما به یک مترجم یا کامپایلر نیاز داریم.

نرم‌افزار منتخب ما **Atmel Studio** است. این نرم‌افزار محیطی حرفه‌ای برای کدنویسی است که فرآیند تبدیل کد به فایل قابل اجرا را انجام می‌دهد. خروجی نهایی این نرم‌افزار فایلی با پسوند **hex** خواهد بود. فایل هگز حاوی کدهای ماشینی است که باید روی حافظه میکروکنترلر نوشته شود (این کار به وسیله پروگرامر انجام میشود).

برای دانلود و نصب **Atmel Studio** کافیست در گوگل جستجو کنید دانلود **Atmel Studio** و اون رو دانلود و نصب کنید (اگر در نصب نیاز به راهنمایی دارید در گوگل جستجو کنید آموزش نصب **Atmel Studio** و از اپارات و یوتیوب کمک بگیرید)

### ۳ محیط شبیه‌سازی پروتئوس

در فرآیند یادگیری و طراحی، بستن مدار واقعی ممکن است زمان‌بر و هزینه‌بر باشد. همچنین احتمال اشتباه در سیم‌کشی و آسیب دیدن قطعات وجود دارد. راه حل هوشمندانه استفاده از شبیه‌ساز است.

نرم‌افزار **Proteus** آزمایشگاهی مجازی در اختیار ما قرار می‌دهد. ما می‌توانیم میکروکنترلر، نمایشگرها، کلیدها و سایر قطعات را در محیط پروتئوس انتخاب کنیم و با سیم‌کشی مجازی به یکدیگر متصل نماییم. سپس فایل هگز تولید شده در **Atmel Studio** را به میکروکنترلر داخل پروتئوس می‌دهیم تا عملکرد مدار را دقیقاً مشابه واقعیت مشاهده کنیم. برای دانلود و نصب پروتئوس کافیت در گوگل جستجو کنید دانلود پروتئوس و اون رو دانلود و نصب کنید (اگر در نصب نیاز به راهنمایی دارید در گوگل جستجو کنید آموزش نصب پروتئوس و از اپارات و یوتیوب کمک بگیرید)

### ۴ ساختار کلی یک برنامه AVR

برای شروع کدنویسی باید با اسکلت‌بندی اصلی برنامه آشنا شویم. هر پروژه‌ای که در AVR Studio ایجاد می‌کنیم ساختاری مشابه کد زیر دارد. بیا این ساختار را تحلیل کنیم.

```
1. /*
2.  * main.c
3.  * Description: Basic structure of AVR C Code
4.  */
5.
6. #include <avr/io.h> // Standard AVR header
7.
8. int main(void)
9. {
10.     // Initialization code goes here
11.
12.     while (1)
13.     {
14.         // Main loop code goes here
15.     }
16. }
```

تحلیل بخش‌های برنامه:

- **هدر فایل avr/io.h** مهم‌ترین بخش برنامه است. این فایل نام پورت‌ها و رجیسترهای میکروکنترلر را در اختیار کامپایلر قرار می‌دهد. نوع میکروکنترلر هنگام ساخت پروژه در **Atmel Studio** مشخص می‌شود و کامپایلر بر اساس آن، فایل رجیستر مخصوص همان تراشه را از داخل **avr/io.h** بارگذاری می‌کند. بدون این هدر، کامپایلر رجیسترهایی مانند **PORTA** یا **DDRB** را نخواهد شناخت.
- **تابع main** مانند تمام برنامه‌های زبان C، اجرای کد از تابع **main** آغاز می‌شود. تنظیمات اولیه سیستم معمولاً در ابتدای این تابع و قبل از حلقه اصلی نوشته می‌شود.
- **حلقه بی‌نهایت while 1** اینجا تفاوت اصلی برنامه‌نویسی سیستم‌های نهفته با برنامه‌نویسی کامپیوتر مشخص می‌شود. در کامپیوتر، برنامه اجرا می‌شود، نتیجه را نمایش می‌دهد و سپس پایان می‌یابد. اما یک میکروکنترلر نباید هرگز متوقف شود. برای مثال سیستم ترمز یک خودرو یا کنترل‌کننده دما باید همواره فعال باشد. به همین دلیل ما از یک حلقه بی‌نهایت **while(1)** استفاده می‌کنیم. کدهایی که درون این حلقه نوشته می‌شوند تا زمانی که جریان برق متصل باشد، بارها و بارها تکرار خواهند شد.

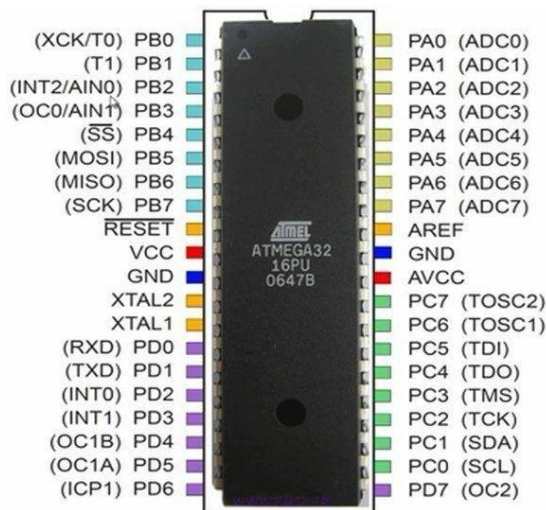
**نکته:** وقتی داخل **Atmel studio** پروژه جدید می‌سازید خودش یه فایل **main.c** داخل پوشه **src** می‌سازه اینجا جاییه که ما کدهای خودمون رو مینویسیم.

**نکته:** برای کامپایل کردن کدها داخل **Atmel studio** دکمه **f7** رو فشار میدیم و کد ما کامپایل و تبدیل به **hex** میشه و این فایل **hex** رو میایم با دبل کلیک روی **atmega32** میایم و بهش میدیم که اجراش کنه



## فصل ۲: کالبدشکافی سخت افزار ATmega32

در فصل قبل ابزارهای نرم‌افزاری را نصب کردیم. اکنون نوبت آن است که خودِ میکروکنترلر را بشناسیم. ما در این دوره از مدل **ATmega32** استفاده می‌کنیم که یکی از کامل‌ترین و محبوب‌ترین اعضای خانواده AVR است. این تراشه یک بسته‌بندی ۴۰ پایه دارد و امکانات بسیار زیادی را در خود جای داده است.



## ۱ بررسی پایه‌ها و ساختار فیزیکی

وقتی به شکل ظاهری ATmega32 نگاه می‌کنیم ۴۰ پایه فلزی می‌بینیم. هر کدام از این پایه‌ها وظیفه خاصی دارند. برای راحتی کار مهندسان این پایه‌ها را گروه‌بندی کرده‌اند.

**پایه‌های تغذیه:** میکروکنترلر یک قطعه الکترونیکی است و برای روشن شدن نیاز به انرژی دارد.

- **VCC** این پایه به قطب مثبت منبع تغذیه وصل می‌شود. ولتاژ استاندارد کاری معمولاً ۵ ولت است.
  - **GND** این پایه به قطب منفی یا زمین مدار متصل می‌شود.
- پورت‌های ورودی و خروجی:** مهم‌ترین بخش برای ما همین پورت‌ها هستند. در ATmega32 ما ۴ پورت اصلی داریم که نام‌هایشان A و B و C و D است. هر پورت شامل ۸ پایه است. دلیل ۸ تایی بودن آن‌ها این است که میکروکنترلر ما ۸ بیتی است و هر پورت یک بایت کامل را تشکیل می‌دهد.
- **Port A:** پایه‌های ۳۳ تا ۴۰
  - **Port B:** پایه‌های ۱ تا ۸
  - **Port C:** پایه‌های ۲۲ تا ۲۹
  - **Port D:** پایه‌های ۱۴ تا ۲۱
- نکته مهم این است که این پایه‌ها **دو طرفه** هستند. یعنی ما می‌توانیم در برنامه تعیین کنیم که یک پایه خاص اطلاعات را به بیرون بفرستد یعنی خروجی باشد یا اطلاعات را از محیط بگیرد یعنی ورودی باشد.

## ۲ سیستم کلاک و نوسان‌ساز

هر پردازنده‌ای برای اینکه کار کند نیاز به یک ضربان قلب دارد. به این ضربان **کلاک پالس** می‌گویند. هرچه این ضربان تندتر باشد سرعت اجرای دستورات بیشتر است. در AVR ما دو راه برای تامین این کلاک داریم:

۱. **نوسان‌ساز داخلی:** خود میکروکنترلر در داخلش یک مدار تولید کلاک دارد که معمولاً روی ۱ مگاهرتز یا ۸ مگاهرتز تنظیم شده است. برای پروژه‌های ساده همین کافی است و نیاز به قطعه اضافی نداریم.

۲. کریستال خارجی: اگر بخواهیم کارهای دقیق زمانی انجام دهیم یا با سرعت بالا به کامپیوتر متصل شویم نیاز به دقت بالایی داریم. در این حالت از یک قطعه به نام کریستال کوارتز استفاده می‌کنیم. این قطعه به پایه‌های شماره ۱۲ و ۱۳ که XTAL1 و XTAL2 نام دارند وصل می‌شود. معمولاً دو خازن کوچک هم برای نویزگیری در کنار آن قرار می‌گیرد.

### ۳ پایه ریست یا شروع مجدد

گاهی لازم است میکروکنترلر را مجبور کنیم که برنامه را از خط اول دوباره اجرا کند. دقیقاً مثل دکمه ریست روی کیس کامپیوتر. پایه شماره ۹ در ATmega32 مخصوص همین کار است و RESET نام دارد. این پایه اصطلاحاً **Active Low** است. یعنی در حالت عادی باید به برق مثبت ۵ ولت وصل باشد تا میکروکنترلر کار کند. هر لحظه که ما این پایه را به منفی یا زمین وصل کنیم میکروکنترلر متوقف شده و به محض رها کردن دوباره از اول شروع به کار می‌کند. در مدارها معمولاً یک کلید فشاری و یک مقاومت پول‌آپ روی این پایه قرار می‌دهند.

### ۴ انواع حافظه در AVR

داخل این تراشه کوچک سه نوع حافظه مختلف وجود دارد که هر کدام کاربرد متفاوتی دارند. درک تفاوت این‌ها برای یک برنامه نویس حیاتی است.

۱. حافظه فلش یا **Flash Memory** اینجا محل ذخیره شدن کدهای برنامه ماست. وقتی در کامپیوتر فایل هگز می‌سازیم و آن را روی میکرو می‌ریزیم در واقع داریم روی این حافظه می‌نویسیم.

- ویژگی مهم: با قطع برق پاک نمی‌شود.
- ظرفیت در ATmega32 برابر با ۳۲ کیلوبایت است که برای هزاران خط کد کافیست.

۲. **حافظه: SRAM** این حافظه میز کار پردازنده است. تمام متغیرها که در زبان C تعریف

می‌کنیم مثل `int a` یا آرایه‌ها در این حافظه ساخته می‌شوند.

- ویژگی مهم: با قطع برق تمام اطلاعات آن از بین می‌رود. سرعت نوشتن و خواندن در آن بسیار بالا است.

۳. **حافظه: EEPROM** این یک حافظه خاص برای ذخیره اطلاعات دائمی است. فرض کنید

می‌خواهیم تنظیمات دستگاه یا رمز عبور را ذخیره کنیم تا اگر برق رفت پاک نشود. ما نمی‌توانیم این‌ها را در فلش بریزیم چون فلش فقط موقع پروگرام کردن نوشته می‌شود. پس از EEPROM استفاده می‌کنیم.

- ویژگی مهم: با قطع برق پاک نمی‌شود ولی سرعت نوشتن در آن پایین است و توسط برنامه قابل خواندن و نوشتن است.

نکته : داخل Atmel Studio زمان ساخت پروژه AT mega 32A رو انتخاب کنید.

## فصل ۳: جادوی رجیسترها

در فصل قبل فهمیدیم که میکروکنترلر پورت‌هایی دارد که به دنیای بیرون وصل هستند. اما سوال اصلی اینجاست که پردازنده مرکزی یا CPU چگونه با این پایه‌ها حرف می‌زند؟ پاسخ یک کلمه است: **رجیستر** یا **Register**

### ۱ مفهوم رجیستر و نقش آن

فرض کن داخل میکروکنترلر یک اتاق فرمان وجود دارد. روی دیوار این اتاق تعدادی کلید برق قرار دارد. وقتی CPU یکی از این کلیدها را روشن می‌کند، همزمان برق یکی از پایه‌های بیرونی میکروکنترلر هم وصل می‌شود. به این ردیف کلیدها **رجیستر** می‌گوییم. در زبان C رجیسترها دقیقاً مثل متغیرها هستند. یعنی یک نام دارند و ما می‌توانیم داخل آن‌ها عدد بریزیم. اما تفاوتشان با متغیر معمولی این است که هر بیت از این متغیر به یک سیم فیزیکی در سخت‌افزار وصل شده است. از آنجایی که میکروکنترلر ما ۸ بیتی است تمام رجیسترهای ما هم ۸ بیت گنجایش دارند. یعنی از بیت شماره ۰ تا بیت شماره ۷.

### ۲ سه تفنگدار پورت‌ها DDR و PORT و PIN

برای کنترل هر پورت در AVR سه رجیستر مخصوص وجود دارد. یعنی برای پورت A سه رجیستر و برای پورت B هم سه رجیستر جداگانه داریم. بیایید وظیفه هر کدام را بررسی کنیم.

۱. رجیستر جهت‌نما یا **DDR** نام کامل این رجیستر **Data Direction Register** است. این رجیستر تعیین می‌کند که یک پایه خاص قرار است **فرستنده (خروجی)** باشد یا **گیرنده (ورودی)**.

- اگر در بیتی از این رجیستر عدد ۱ بنویسیم آن پایه **خروجی** می‌شود. مثل وقتی که می‌خواهیم **LED** روشن کنیم.

- اگر در بیتی از این رجیستر عدد ۰ بنویسیم آن پایه **ورودی** می‌شود. مثل وقتی که می‌خواهیم وضعیت یک کلید را بخوانیم.

۲. رجیستر داده یا **PORT** این رجیستر وظیفه تعیین سطح ولتاژ خروجی را دارد. البته فقط زمانی که پایه را خروجی تعریف کرده باشیم.

- نوشتن عدد ۱ در این رجیستر باعث می‌شود ولتاژ پایه ۵ ولت شود.

- نوشتن عدد ۰ در این رجیستر باعث می‌شود ولتاژ پایه ۰ ولت شود.

۳. رجیستر ورودی یا **PIN** نام کامل این رجیستر **Port Input Pins** است. زمانی که پایه را به صورت ورودی تعریف کردیم، برای اینکه بفهمیم الان ولتاژ روی پایه وجود دارد یا خیر باید این رجیستر را بخوانیم. ما نمی‌توانیم در این رجیستر چیزی بنویسیم و فقط خواندنی است.

### ۳ منطق باینری و بیت‌وايز

ما در زبان C یاد گرفتیم چطور با اعداد دهدهی کار کنیم. اما اینجا باید با بیت‌ها حرف بزنیم. فرض کنید می‌خواهیم فقط پایه شماره ۳ از پورت B را خروجی کنیم و بقیه پایه‌ها دست‌نخورده باقی بمانند. اگر بنویسیم  $DDR_B = 8$  کار انجام می‌شود اما تمام پایه‌های دیگر پورت B صفر می‌شوند و تنظیماتشان به هم می‌ریزد. برای حل این مشکل از **عملگرهای** **بیتی** استفاده می‌کنیم.

**عملگر OR** یا **|** برای یک کردن بیت هر بیتی که با ۱ عملیات OR شود نتیجه‌اش ۱ خواهد شد و اگر با ۰ عملیات OR شود تغییری نمی‌کند. پس برای روشن کردن یک بیت خاص از دستور زیر استفاده می‌کنیم:

```
1. // Set bit 3 of PORTB to 1 without changing other bits
2. PORTB = PORTB | 0b00001000;
```

**عملگر شیفت به چپ** یا **<<** نوشتن عدد باینری سخت است. دستور شیفت کار را راحت می‌کند. عبارت  $1 << 3$  یعنی عدد ۱ را بردار و ۳ بار به سمت چپ هل بده. نتیجه می‌شود همان 0b00001000.

**عملگر AND** یا **&** و **نقیض** ~ برای صفر کردن بیت برای اینکه یک بیت خاص را ۰ کنیم و بقیه تغییر نکنند باید آن بیت را با ۰ عملیات AND کنیم و بقیه را با ۱. برای این کار از علامت نقیض یا تیلدا ~ استفاده می‌کنیم.

#### ۴ تعیین جهت پورت‌ها با کدنویسی

حالا بیایید تمام این مفاهیم را در قالب چند مثال واقعی کدنویسی ترکیب کنیم. این‌ها مهم‌ترین خطوط کدی هستند که در فصل‌های آینده مدام می‌بینیم.

مثال ۱: تنظیم پایه ۳ از پورت A به عنوان خروجی می‌خواهیم به میکرو بگوییم پایه PA3 خروجی باشد.

```
1. // Set PA3 as Output
2. DDRA = DDRA | (1 << 3);
3. // Or shorter version:
4. DDRA |= (1 << 3);
```

مثال ۲: روشن کردن پایه ۳ از پورت A حالا که خروجی شد می‌خواهیم برق ۵ ولت را روی آن بفرستیم.

```
1. // Set PA3 High (5V)
2. PORTA |= (1 << 3);
```

مثال ۳: خاموش کردن پایه ۳ از پورت A حالا می‌خواهیم ولتاژ آن را قطع کنیم.

```
1. // Set PA3 Low (0V)
2. PORTA &= ~(1 << 3);
```

مثال ۴: تنظیم پایه ۰ از پورت B به عنوان ورودی می‌خواهیم یک کلید به پایه PB0 وصل کنیم پس باید ورودی باشد.

```
1. // Set PB0 as Input
2. DDRB &= ~(1 << 0);
```

**نکته طلایی:** همیشه به یاد داشته باشید: ۱. اول با **DDR** جهت خیابان را مشخص کنید که یک‌طرفه به بیرون است یا به داخل. ۲. سپس اگر خروجی بود با **PORT** اطلاعات بفرستید. ۳. و اگر ورودی بود با **PIN** اطلاعات بخوانید.



# فصل ۴: اولین پروژه عملی؛ چشمک زدن LED

در فصل‌های قبل با ابزار کار آشنا شدیم و یاد گرفتیم که رجیسترها فرماندهان اصلی میکروکنترلر هستند. حالا وقت آن رسیده که آستین‌ها را بالا بزنیم و اولین پروژه واقعی خودمان را بسازیم. در دنیای برنامه نویسی کامپیوتر اولین برنامه همیشه چاپ کردن Hello World است، اما در دنیای الکترونیک و میکروکنترلرها، اولین پروژه همیشه روشن و خاموش کردن یک LED یا همان پروژه Blinking LED است.

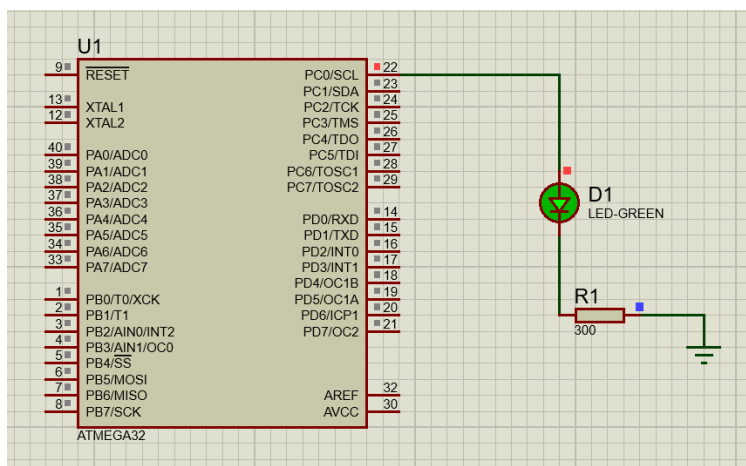
## صورت مسئله

ما می‌خواهیم یک LED را به یکی از پایه‌های میکروکنترلر ATmega32 وصل کنیم و برنامه‌ای بنویسیم که این لامپ کوچک هر ۵۰۰ میلی‌ثانیه یک بار روشن و خاموش شود.

## ۱: طراحی سخت‌افزار در پروتئوس

قبل از کدنویسی باید مدار را در نرم‌افزار پروتئوس ببندیم. برای این کار قطعات زیر را نیاز داریم:

- میکروکنترلر ATmega32
  - یک عدد LED رنگ دلخواه
  - یک عدد مقاومت ۲۲۰ اهم یا ۳۳۰ اهم
- نحوه اتصال:** ما پورت C و پایه شماره صفر آن یعنی PC0 را برای این کار انتخاب می‌کنیم.
۱. پایه مثبت یا آند LED را به پایه PC0 میکروکنترلر وصل کنید.
  ۲. پایه منفی یا کاتد LED را به یک سر مقاومت وصل کنید.
  ۳. سر دیگر مقاومت را به زمین یا GND وصل کنید.



نکته مهم اینجاست که مقاومت نقش محافظ را بازی می‌کند. اگر LED را مستقیم به میکروکنترلر وصل کنیم، جریان زیاد باعث سوختن LED یا آسیب دیدن پورت میکروکنترلر می‌شود.

## ۲: چالش زمان و تابع تأخیر

میکروکنترلرها سرعت بسیار بالایی دارند. اگر ما در یک خط کد دستور روشن شدن بدهیم و در خط بعدی دستور خاموش شدن، این اتفاق آنقدر سریع می‌افتد که چشم انسان قادر به دیدن آن نیست و ما LED را همیشه روشن اما با نور کم می‌بینیم.

برای حل این مشکل باید بین دستور روشن و خاموش کردن، یک وقفه یا Delay ایجاد کنیم. یعنی به CPU بگوییم بعد از روشن کردن لامپ، برای مدتی هیچ کاری انجام ندهد و صبر کند.

برای استفاده از تأخیر در زبان C و کامپایلر Atmel Studio باید از کتابخانه مخصوصی به نام `util/delay.h` استفاده کنیم. همچنین باید به کامپایلر بگوییم سرعت کلاک میکروکنترلر ما چقدر است تا بتواند زمان دقیق را محاسبه کند.

### ۳: کدنویسی در Atmel Studio

حالا بیایید کدهای این پروژه را بنویسیم. در این کد از مفاهیم رجیستر DDR برای تعیین جهت و رجیستر PORT برای مقداردهی استفاده می‌کنیم که در فصل قبل یاد گرفتیم.

```
1. /*
2.  * main.c
3.  *
4.  * Description: Toggling an LED connected to PC0 every 500ms
5.  * Author: MahdiGhaemi
6.  */
7.
8. // Define CPU frequency as 1MHz (Default for ATmega32)
9. // This must be defined BEFORE including delay.h
10. #define F_CPU 1000000UL
11.
12. #include <avr/io.h>      // Standard AVR header for registers
13. #include <util/delay.h>  // Header for delay functions like _delay_ms()
14.
15. int main(void)
16. {
17.     // Configuration Step
18.     // We want PC0 to be an Output pin.
19.     // Based on Chapter 3: set DDR bit to 1 for Output
20.     DDRC |= (1 << 0);    // Set PC0 as Output
21.
22.     // Main Loop
23.     // Code inside while(1) runs forever [cite: 92]
24.     while (1)
25.     {
26.         // 1. Turn on the LED
27.         // Set PC0 to High (5V) using PORT register [cite: 225]
28.         PORTC |= (1 << 0);
29.
30.         // 2. Wait for 500 milliseconds
31.         _delay_ms(500);
32.
33.         // 3. Turn off the LED
34.         // Set PC0 to Low (0V) using PORT register [cite: 226]
35.         // We use AND logic with NOT (~) to clear the bit
36.         PORTC &= ~(1 << 0);
37.
38.         // 4. Wait for 500 milliseconds again
39.         _delay_ms(500);
40.     }
41. }
```

#### ۴: تحلیل خط به خط برنامه

بخش هدر فایل‌ها: در ابتدای برنامه F\_CPU را تعریف کردیم. عبارت 1000000UL یعنی فرکانس ۱ مگاهرتز که سرعت پیش‌فرض ATmega32 است. اگر این عدد را ننویسیم، توابع تأخیر درست کار نمی‌کنند. سپس کتابخانه util/delay.h را اضافه کردیم تا بتوانیم از دستور delay\_ms استفاده کنیم.

تنظیمات اولیه: در داخل تابع main و قبل از حلقه اصلی، جهت پایه را مشخص کردیم. همان‌طور که در فصل ۳ خواندیم، برای اینکه پایه خروجی باشد باید در رجیستر DDR عدد ۱ بنویسیم. دستور  $DDRC |= (1 << 0)$  دقیقاً بیت شماره صفر از پورت C را یک می‌کند بدون اینکه به بقیه پایه‌ها کاری داشته باشد.

حلقه بی‌نهایت: درون while(1) چهار مرحله مدام تکرار می‌شوند:

۱. روشن کردن: با دستور  $PORTC |= (1 << 0)$  پایه PC0 را یک می‌کنیم تا ولتاژ ۵

ولت شود و LED روشن شود.

۲. صبر کردن: تابع delay\_ms(500) اجرای برنامه را نیم ثانیه متوقف می‌کند.

۳. خاموش کردن: با دستور  $PORTC \&= \sim(1 << 0)$  پایه PC0 را صفر می‌کنیم.

علامت ~ بیت را معکوس می‌کند و & باعث می‌شود فقط همان بیت صفر شود.

۴. صبر کردن: دوباره نیم ثانیه صبر می‌کنیم تا خاموش بودن LED هم دیده شود.

#### ۵: تست نهایی

حالا کافیت در Atmel Studio دکمه Build را بزنید تا فایل هگز (.hex) ساخته شود. سپس در نرم‌افزار پروتئوس روی میکروکنترلر دابل کلیک کنید و در بخش Program File آدرس فایل هگز ساخته شده را بدهید. با زدن دکمه Play در پروتئوس، می‌بینید که LED شروع به چشمک زدن می‌کند. تبریک می‌گویم، شما اولین سیستم نهفته خود را ساختید.

# فصل ۵: خواندن ورودی‌ها؛ تعامل با کلیدها

در فصل قبل یاد گرفتیم چگونه با روشن کردن LED به دنیای بیرون فرمان بدهیم. اما یک سیستم هوشمند فقط دستور نمی‌دهد بلکه باید بتواند محیط اطرافش را هم حس کند. در این فصل یاد می‌گیریم چگونه وضعیت یک کلید یا Push Button را بخوانیم. این پایه و اساس کار با کیبوردها و سنسورهاست.

## مفهوم ورودی و مشکل نویز

وقتی پایه‌ای را به عنوان ورودی تعریف می‌کنیم میکروکنترلر منتظر است تا ببیند ولتاژ روی این پایه ۵ ولت است یا ۰ ولت. اگر یک کلید را مستقیم به پایه میکروکنترلر وصل کنیم وقتی کلید باز است پایه به هیچ جا وصل نیست. در این حالت پایه اصطلاحاً "معلق" یا Floating است. پایه معلق مثل یک آنتن عمل می‌کند و نویزهای محیط را می‌گیرد. میکروکنترلر گیج می‌شود و گاهی ۱ می‌خواند و گاهی ۰.

**راه حل: مقاومت پول‌آپ (Pull-up)** برای حل این مشکل از یک مقاومت استفاده می‌کنیم که پایه را به برق ۵ ولت متصل نگه می‌دارد. در این حالت وقتی کلید فشرده نیست میکروکنترلر عدد ۱ را می‌خواند. وقتی کلید را فشار می‌دهیم پایه به زمین یا GND وصل می‌شود و میکروکنترلر عدد ۰ را می‌خواند. خبر خوب این است که خانواده AVR درون خودشان این مقاومت‌ها را دارند و نیازی نیست در مدار شلوغ‌کاری کنیم. فقط کافیست با کدنویسی آن‌ها را فعال کنیم.

## صورت مسئله پروژه

ما می‌خواهیم مداری بسازیم که یک کلید به پایه PA0 و یک LED به پایه PC0 متصل باشد.

- تا زمانی که دستان روی کلید است LED روشن بماند.
- به محض برداشتن دست از روی کلید LED خاموش شود.

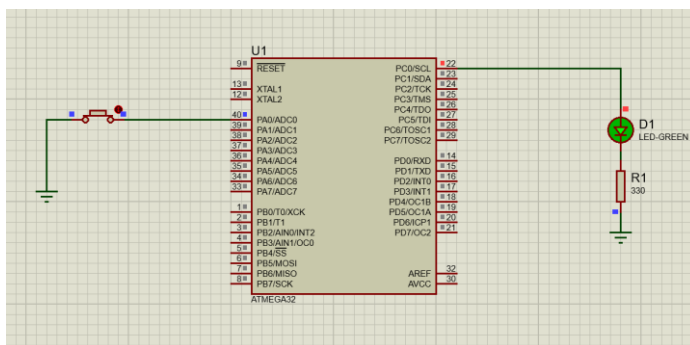
## ۱: بستن مدار در پروتئوس

برای این پروژه به قطعات زیر نیاز داریم:

- میکروکنترلر ATmega32
- یک عدد LED و مقاومت ۳۳۰ اهم متصل به پایه PC0 مانند فصل قبل
- یک عدد دکمه فشاری یا Button

اتصال دکمه:

۱. یک سر دکمه را به پایه شماره ۴۰ یعنی PA0 وصل کنید.
۲. سر دیگر دکمه را به زمین یا GND وصل کنید. توجه کنید که چون از مقاومت داخلی میکروکنترلر استفاده می‌کنیم نیازی به مقاومت اضافی برای دکمه در پروتئوس نداریم.



## ۲: کدنویسی و فعال سازی مقاومت داخلی

نکته طلایی این فصل در رجیستر PORT نهفته است. ما در فصل ۳ یاد گرفتیم که اگر پایه خروجی باشد رجیستر PORT ولتاژ را تعیین می‌کند. اما اگر پایه ورودی باشد نوشتن عدد ۱ در رجیستر PORT باعث فعال شدن مقاومت پول آپ داخلی می‌شود.

بنابراین منطق برنامه ما به این صورت است

۱. پایه کلید ورودی باشد DDR صفر شود

۲. مقاومت داخلی کلید فعال شود PORT یک شود.

۳. پایه LED خروجی باشد.

۴. در حلقه اصلی وضعیت پایه کلید را از روی رجیستر PIN بخوانیم.

### کد برنامه در Atmel Studio

```
1. /*
2.  * main.c
3.  *
4.  * Description: Turn on LED on PC0 when Button on PA0 is pressed
5.  * Note: Using Internal Pull-up Resistor
6.  * Author: MahdiGhaemi
7.  */
8.
9. #include <avr/io.h>
10.
11. int main(void)
12. {
13.     // --- Configuration ---
14.
15.     // 1. Setup LED (PC0)
16.     // Set PC0 as Output
17.     DDRC |= (1 << 0);
18.
19.     // 2. Setup Button (PA0)
20.     // Set PA0 as Input (Default is 0, but good to be explicit)
21.     DDRA &= ~(1 << 0);
22.
23.     // Enable Internal Pull-up Resistor for PA0
24.     // We write 1 to PORT register while pin is Input
25.     PORTA |= (1 << 0);
26.
27.     // --- Main Loop ---
28.     while (1)
29.     {
30.         /* Check if button is pressed.
31.          Since we use Pull-up, the logic is inverted (Active Low):
32.          - Not Pressed = Voltage is 5V (Logic 1)
33.          - Pressed     = Voltage is 0V (Logic 0)
34.
35.          We read the PINA register to check the voltage.
36.         */
```

```

37.
38.     if ( (PINA & (1 << 0)) == 0 )
39.     {
40.         // Button is pressed (Logic 0)
41.         // Turn ON LED
42.         PORTC |= (1 << 0);
43.     }
44.     else
45.     {
46.         // Button is released (Logic 1)
47.         // Turn OFF LED
48.         PORTC &= ~(1 << 0);
49.     }
50. }
51. }

```

### ۳: تحلیل منطق شرط IF

- شاید قسمت شرط if کمی گیج کننده باشد. بیایید بازش کنیم. ما از دستور  $(PINA \& (1 \ll 0))$  استفاده کردیم. این دستور یعنی برو و فقط بیت شماره صفر از رجیستر ورودی A را نگاه کن و بقیه بیت ها را نادیده بگیر.
- **حالت فشرده:** چون کلید به زمین وصل می شود ولتاژ ۰ می شود. پس نتیجه عملیات بالا صفر خواهد بود. شرط  $== 0$  صحیح می شود و دستور روشن شدن LED اجرا می گردد.
  - **حالت رها شده:** چون مقاومت پول آپ فعال است ولتاژ ۵ ولت یا منطق ۱ است. پس نتیجه صفر نیست و برنامه به قسمت else می رود و LED را خاموش می کند. این روش که در آن صفر بودن به معنی فعال بودن است را منطق **Active Low** می گویند و در صنعت بسیار رایج است.



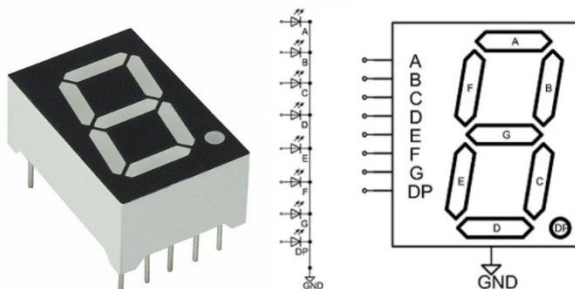
## فصل ۶: بازی با اعداد؛ راه اندازی سون سگمنت (7-Segment)

در فصل‌های قبل یاد گرفتیم چطور یک چراغ LED تکی را روشن و خاموش کنیم. حالا بیایید کمی فراتر برویم. تصور کنید ۷ عدد LED را به شکل عدد ۸ انگلیسی کنار هم بچینیم. به این قطعه سون سگمنت یا ۷ Segment می‌گویند.

این نمایشگر ارزان‌ترین، جان‌سخت‌ترین و ساده‌ترین راه برای نشان دادن اعداد در پروژه‌های الکترونیکی است؛ از تایمر چراغ‌های راهنمایی سر چهارراه‌ها گرفته تا ساعت‌های دیجیتال قدیمی و نمایشگر طبقات آسانسور.

در این فصل می‌خواهیم یک پروژه کلاسیک بسازیم: یک شمارنده که اعداد ۰ تا ۹ را به ترتیب نمایش دهد.

### 7-Segment Display



## کالبدشکافی سون سگمنت

نام‌گذاری این ۷ خط نورانی قانون و استاندارد جهانی دارد. آن‌ها را با حروف کوچک انگلیسی از **a** تا **g** نام‌گذاری می‌کنند:

- **a:** خط افقی بالایی
- **b:** خط عمودی بالا سمت راست
- **c:** خط عمودی پایین سمت راست
- **d:** خط افقی پایینی
- **e:** خط عمودی پایین سمت چپ
- **f:** خط عمودی بالا سمت چپ
- **g:** خط وسط
- **dp:** نقطه اعشار (معمولاً گوشه پایین سمت راست)

نکته مهم سخت‌افزاری: کاتد مشترک یا آند مشترک؟

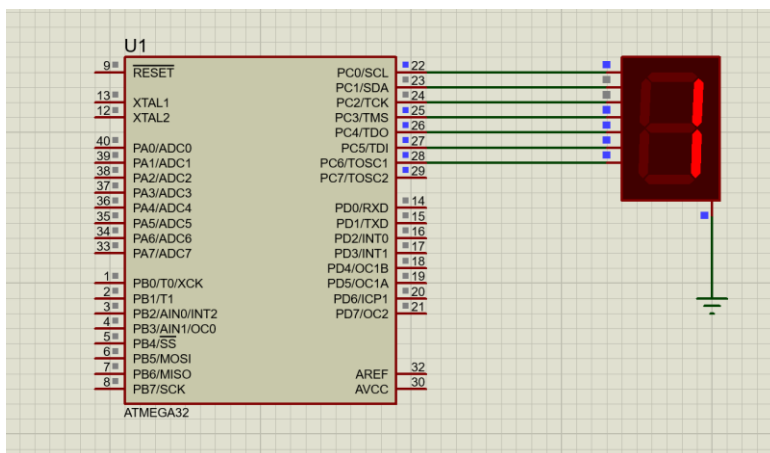
سون سگمنت‌ها در دو مدل ساخته می‌شوند. ما در این دوره از مدل کاتد مشترک (Common Cathode) استفاده می‌کنیم.

- **کاتد مشترک:** یعنی پایه منفی (کاتد) تمام LED های داخلی به هم وصل شده و باید به زمین (GND) متصل شود. بنابراین برای روشن کردن هر خط، باید به پایه مربوط به آن ولتاژ مثبت (۵ ولت) یا همان منطق ۱ بدهیم.

## نگاشت بیت‌ها؛ تبدیل نقاشی به صفر و یک

چالش اصلی اینجا است که میکروکنترلر "شکل" عدد ۲ یا ۵ را نمی‌شناسد. میکروکنترلر فقط بایت و بیت می‌فهمد. ما باید یک "مترجم" بسازیم. بهترین راه این است که پورت میکروکنترلر را مستقیم به پایه‌های سون سگمنت وصل کنیم. فرض کنید پورت C را انتخاب کرده‌ایم:

- بیت ۰ -> (PC0) وصل شود به **a** ->
- بیت ۱ -> (PC1) وصل شود به **b** ->
- ...
- بیت ۶ -> (PC6) وصل شود به **g** ->
- بیت ۷ -> (PC7) وصل شود به **dp** -> نقطه



### چرا کد باینری (دودویی) بهتر از هگزادسیمال است؟

وقتی می‌خواهید شکل یک عدد را طراحی کنید، استفاده از مبنای ۲ (صفر و یک) بسیار راحت‌تر است چون دقیقاً وضعیت خاموش/روشن بودن لامپ‌ها را نشان می‌دهد. بیا یک کد عدد "صفر" را بسازیم:

برای نمایش ۰، همه چراغ‌ها دور تا دور باید روشن باشند و فقط خط وسط (g) و نقطه (dp) خاموش باشند.

| نام سگمنت | dp    | g     | f    | e    | d    | c    | b    | a    |
|-----------|-------|-------|------|------|------|------|------|------|
| وضعیت     | خاموش | خاموش | روشن | روشن | روشن | روشن | روشن | روشن |

| نام سگمنت | dp | g | f | e | d | c | b | a |
|-----------|----|---|---|---|---|---|---|---|
| مقدار بیت | 0  | 0 | 1 | 1 | 1 | 1 | 1 | 1 |

پس کد باینری عدد صفر می‌شود: ۰b00111111  
پیشوند 0b به کامپایلر می‌گوید که این عدد مبنای ۲ است.

### مرحله ۱: بستن مدار در پروتئوس

قبل از کدنویسی، باید سخت‌افزار را آماده کنیم.

قطعات مورد نیاز:

۱. میکروکنترلر ATmega32

۲. نمایشگر سون سگمنت مدل 7SEG-COM-CATHODE رنگ آن مهم نیست.

روش سیم‌کشی:

۱. پایه PC0 میکروکنترلر را به پایه a نمایشگر وصل کنید.

۲. پایه PC1 را به پایه b وصل کنید.

۳. این روند را ادامه دهید تا PC6 که به پایه g وصل می‌شود.

۴. پایه PC7 را به پایه dp (نقطه) وصل کنید (یا آزاد بگذارید).

۵. نکته حیاتی: نمایشگر یک پایه مشترک (Common) دارد که معمولاً در بالا یا پایین

قطعه است. چون مدل ما کاتد مشترک است، این پایه را حتماً باید به GND زمین

وصل کنید. اگر این کار را نکنید، هیچ چیزی روشن نمی‌شود.

### مرحله ۲: کدنویسی شمارنده با استفاده از آرایه

در این برنامه از یک "آرایه" به عنوان "جدول تبدیل" استفاده می‌کنیم. اندیس آرایه همان عددی است که می‌خواهیم نمایش دهیم و محتوای آن، کد باینری مربوطه است.

```
1. /*
2.  * main.c
3.  *
```

```

4.  * Description: Counter 0 to 9 using Binary Look-up Table
5.  * Hardware: Common Cathode 7-Segment on PORTC
6.  * Wiring: PC0->a, PC1->b, ..., PC6->g, PC7->dp
7.  * Author: MahdiGhaemi
8.  */
9.
10. #define F_CPU 1000000UL // Set CPU Frequency to 1 MHz
11. #include <avr/io.h>
12. #include <util/delay.h>
13.
14. int main(void)
15. {
16.     // --- Configuration ---
17.
18.     // Set all 8 pins of PORTC as Output
19.     // 0xFF equals 11111111 in binary
20.     DDRC = 0xFF;
21.
22.     // --- Look-up Table Setup ---
23.     // This array holds the codes required to display numbers 0 to 9.
24.     // Bit order: 0b(dp)(g)(f)(e)(d)(c)(b)(a)
25.     // 1 = ON, 0 = OFF
26.
27.     unsigned char segment_codes[10] = {
28.         0b00111111, // Number 0: All ON except g and dp
29.         0b00000110, // Number 1: Only b and c are ON
30.         0b01011011, // Number 2: a, b, d, e, g are ON
31.         0b01001111, // Number 3: a, b, c, d, g are ON
32.         0b01100110, // Number 4: b, c, f, g are ON
33.         0b01101101, // Number 5: a, c, d, f, g are ON
34.         0b01111101, // Number 6: All ON except b
35.         0b00000111, // Number 7: Only a, b, c are ON
36.         0b01111111, // Number 8: All segments ON
37.         0b01101111 // Number 9: All ON except e
38.     };
39.
40.     // Define loop counter variable
41.     int i;
42.
43.     // --- Main Loop ---
44.     while (1)
45.     {
46.         // A for loop that counts from 0 to 9
47.         for (i = 0; i < 10; i++)
48.         {
49.             // Use 'i' as the index to access the array
50.             // Extract the corresponding binary code and write it to PORTC
51.             PORTC = segment_codes[i];
52.
53.             // Wait for 1 second to make the number visible
54.             _delay_ms(1000);
55.         }
56.
57.         // When the for loop finishes (reaches 9), since we are inside
while(1)

```

```

58.      // the program loops back to the top and counting starts from 0
again.
59.    }
60. }

```

## تحلیل نهایی و عمیق برنامه

بیا یاد کدی که نوشتیم را زیر ذره بین ببریم تا مفاهیم مهندسی آن را درک کنیم:

### ۱. قدرت جدول جستجو (Look-up Table)

مهم‌ترین تکنیک استفاده شده در این فصل، `segment_codes` است. در سیستم‌های نهفته، ما ترجیح می‌دهیم محاسبات را "پیش‌پردازش" کنیم. به جای اینکه میکروکنترلر هر بار فکر کند "برای نمایش عدد ۳ کدام پایه‌ها را روشن کنم؟"، ما پاسخ‌ها را از قبل محاسبه کرده و در یک آرایه چیده‌ایم. این کار سرعت اجرای برنامه را بالا می‌برد و کد را بسیار تمیز می‌کند.

### ۲. نگاشت حافظه به سخت‌افزار (Memory Mapped I/O)

وقتی دستور `PORTC = segment_codes[i]` اجرا می‌شود، اتفاق جالبی می‌افتد. پردازنده یک بایت داده (۸ بیت) را از حافظه SRAM (جایی که آرایه هست) برمی‌دارد و آن را مستقیماً روی ۸ پین فیزیکی میکروکنترلر کپی می‌کند. این عملیات به صورت موازی (Parallel) انجام می‌شود؛ یعنی هر ۸ چراغ دقیقاً در یک لحظه روشن یا خاموش می‌شوند و هیچ تأخیری بین روشن شدن سگمنت `a` و `b` وجود ندارد.

### ۳. جادوی اعداد باینری (Binary Literals)

استفاده از `0b` کد ما را "گرافیکی" کرد. اگر دقت کنید، الگوی یک‌ها و صفرها در کد، شبیه به الگوی روشن و خاموش شدن لامپ‌هاست. این روش دیباگ کردن را آسان می‌کند. مثلاً اگر سگمنت وسط (`g`) روشن نمی‌شد، کافی بود بیت ششم تمام اعداد را چک کنید. اگر از هگزادسیمال مثلاً `0x5B` استفاده می‌کردیم، پیدا کردن این ایراد نیاز به ماشین حساب و تبدیل مبنا داشت.

### ۴. مدیریت پورت (DDR vs PORT)

در ابتدای برنامه  $DDRC = 0xFF$  را نوشتیم. اگر این خط را فراموش می کردید، چه می شد؟ میکروکنترلر به صورت پیش فرض پایه ها را "ورودی" در نظر می گیرد. اگر شما روی پورت ورودی داده بنویسید در PORTC عدد بریزید، به جای روشن شدن LED ها، مقاومت های داخلی (Pull-up) فعال می شدند و نور بسیار ضعیفی می دیدید یا اصلاً چیزی روشن نمی شد. پس همیشه اول جهت را تعیین کنید (DDR)، سپس فرمان دهید. (PORT)

# فصل ۷: حرف زدن با کاربر؛ راه اندازی نمایشگر LCD

در فصل قبل با سون سگمنت کار کردیم و متوجه محدودیت بزرگ آن شدیم. سون سگمنت فقط می‌تواند اعداد را نشان دهد و برای سیم‌کشی هم تعداد زیادی پایه مصرف می‌کند. اگر بخواهیم یک پیام خطا مثل Error یا یک پیام خوش آمدگویی بنویسیم چه؟ اینجاست که LCD کاراکتری وارد میدان می‌شود. مدلی که ما استفاده می‌کنیم معروف به ۱۶ در ۲ است. یعنی می‌تواند ۱۶ کاراکتر در سطر اول و ۱۶ کاراکتر در سطر دوم نمایش دهد. این نمایشگر استانداردترین ابزار برای ارتباط با کاربر در سیستم‌های نهفته است.

## زبان ال‌سی‌دی؛ پایه RS چیست؟

این نمایشگر یک پردازنده کوچک در دل خود دارد که ما باید با آن حرف بزنیم. پایه‌های D0 تا D7 اتوبان انتقال اطلاعات هستند. اما این اطلاعات دو نوع است:

۱. دستور: (Command) مثلاً "صفحه را پاک کن" یا "برو به خط دوم"
۲. داده: (Data) مثلاً حرف 'A' یا عدد '۵' که باید نمایش داده شود.

تشخیص این دو بر عهده پایه RS یا Register Select است:

- اگر RS=0 باشد میکروکنترلر دارد دستور می‌دهد.
  - اگر RS=1 باشد میکروکنترلر دارد متن می‌فرستد.
- پایه دیگر E یا Enable است که مثل کلید Enter عمل می‌کند. تا وقتی این پایه را یک و سپس صفر نکنیم (یک پالس ندهیم) ال‌سی‌دی اطلاعات روی پایه‌ها را نمی‌خواند.



## مرحله ۱: بستن مدار در پروتئوس

قطعات لازم:

- میکروکنترلر ATmega32
- نمایشگر LM016L همان LCD 16x2

اتصالات (مد ۸ بیتی): برای سادگی در یادگیری ما از ۸ سیم برای انتقال دیتا استفاده

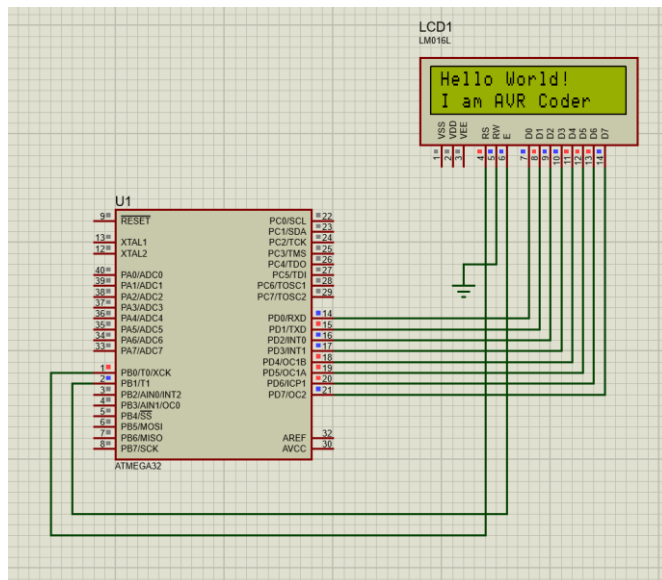
می‌کنیم

۱. پایه‌های D0 تا D7 نمایشگر را به ترتیب به پورت **D** میکروکنترلر PD0 تا PD7 وصل کنید.

۲. پایه RS نمایشگر را به **PB0** وصل کنید.

۳. پایه E نمایشگر را به **PB1** وصل کنید.

۴. پایه RW نمایشگر را به **GND** (زمین) وصل کنید چون ما فقط می‌خواهیم بنویسیم و خواندنی نداریم.



## مرحله ۲: کدنویسی درایور LCD

چون نوشتن دستورات LCD کمی پیچیده است ما توابع مخصوص آن را بالای برنامه اصلی می‌نویسیم تا کد تمیز شود. این همان کاری است که کتابخانه‌ها انجام می‌دهند.

```
1. /*
2.  * main.c
3.  *
4.  * Description: Displaying Name on 16x2 LCD
5.  * Mode: 8-bit Data Mode
6.  * Connection: Data->PORTD, Control->PORTB
7.  * Author: MahdiGhaemi
8.  */
9.
10. #define F_CPU 1000000UL
11. #include <avr/io.h>
12. #include <util/delay.h>
13.
14. // --- Helper Macros ---
15. #define LCD_DATA_PORT PORTD
16. #define LCD_CTRL_PORT PORTB
17. #define RS_PIN 0
18. #define E_PIN 1
19.
20. // --- Function 1: Send Command to LCD ---
21. // Use this to clear screen, move cursor, etc.
22. void LCD_Cmd(unsigned char cmd)
23. {
24.     LCD_DATA_PORT = cmd;           // Put command on data bus
25.     LCD_CTRL_PORT &= ~(1<<RS_PIN); // RS = 0 for Command
26.
27.     // Generate Pulse on E pin (High -> Low)
28.     LCD_CTRL_PORT |= (1<<E_PIN);
29.     _delay_ms(1);
30.     LCD_CTRL_PORT &= ~(1<<E_PIN);
31.     _delay_ms(2); // Wait for LCD to process
32. }
33.
34. // --- Function 2: Send Character to LCD ---
35. // Use this to print letters like 'A', 'b', etc.
36. void LCD_Char(unsigned char data)
37. {
38.     LCD_DATA_PORT = data;           // Put character on data bus
39.     LCD_CTRL_PORT |= (1<<RS_PIN); // RS = 1 for Data
40.
41.     // Generate Pulse on E pin
42.     LCD_CTRL_PORT |= (1<<E_PIN);
43.     _delay_ms(1);
44.     LCD_CTRL_PORT &= ~(1<<E_PIN);
45.     _delay_ms(2);
```

```
46. }
47.
48. // --- Function 3: Initialize LCD ---
49. // Standard startup sequence for 16x2 LCD
50. void LCD_Init()
51. {
52.     DDRD = 0xFF; // Data Port as Output
53.     DDRB = 0x03; // Control Pins (PB0, PB1) as Output
54.
55.     _delay_ms(20); // Wait for power up
56.     LCD_Cmd(0x38); // Mode: 8-bit, 2 lines, 5x7 dots
57.     LCD_Cmd(0x0C); // Display ON, Cursor OFF
58.     LCD_Cmd(0x01); // Clear Display
59.     _delay_ms(2);
60.     LCD_Cmd(0x80); // Move cursor to beginning
61. }
62.
63. // --- Function 4: Send String ---
64. // Print a full sentence
65. void LCD_String(char *str)
66. {
67.     int i;
68.     // Loop through the string until null terminator
69.     for(i=0; str[i] != 0; i++)
70.     {
71.         LCD_Char(str[i]);
72.     }
73. }
74.
75. // --- Main Function ---
76. int main(void)
77. {
78.     LCD_Init(); // Wake up the LCD
79.
80.     // Print first line
81.     LCD_String("Hello World!");
82.
83.     // Move to second line (Command 0xC0)
84.     LCD_Cmd(0xC0);
85.
86.     // Print your name
87.     LCD_String("I am AVR Coder");
88.
89.     while (1)
90.     {
91.         // Nothing to do here, text stays on screen
92.     }
93. }
```

## تحلیل نهایی

این برنامه شاید در نگاه اول طولانی به نظر برسد اما اگر آن را بشکافیم بسیار منطقی است.

۱. **معماری توابع: (Function Architecture)** ما به جای اینکه کدهای تکراری روشن و خاموش کردن پین‌ها را در main بنویسیم، دو تابع پایه به نام LCD\_Cmd و LCD\_Data ساختیم. این توابع "لایه سخت‌افزار" هستند. تابع LCD\_String از این توابع استفاده می‌کند و "لایه نرم‌افزار" را می‌سازد. در مهندسی نرم‌افزار به این کار "لایه انتزاعی" یا Abstraction می‌گویند.

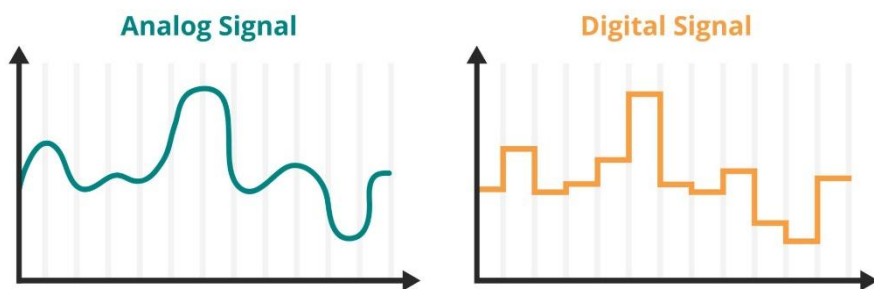
۲. **نقش جدول اسکی: (ASCII)** وقتی شما دستور LCD\_Char('A') را صدا می‌زنید، کامپایلر زبان C به صورت خودکار کد اسکی حرف A یعنی عدد ۶۵ یا ۰x41 را روی پورت D قرار می‌دهد. ال‌سی‌دی هم یک جدول داخلی دارد و می‌داند که برای کد ۶۵ باید پیکسل‌های مربوط به شکل A را روشن کند.

۳. **کارکرد تابع رشته: (String Function)** تابع LCD\_String از خاصیت آرایه‌ای بودن رشته‌ها در C استفاده می‌کند. این تابع کاراکترها را دانه دانه برمی‌دارد و به تابع LCD\_Char می‌دهد تا زمانی که به کاراکتر تهی یا ۰\ برسد که نشانه پایان متن است.

۴. **دستورات جادویی: (Magic Commands)** در تابع Init کدهایی مثل 0x38 دیدید. این‌ها کدهای استاندارد کارخانه سازنده LCD هستند.

- 0x38: یعنی من ۸ سیم دیتا دارم و ۲ خط نمایشگر.
- 0x01: یعنی هرچه روی صفحه هست پاک کن.
- 0xC0: یعنی نشانگر تاپ را به اول خط دوم ببر.

## فصل ۸: پل میان دیجیتال و آنالوگ (ADC)؛ ساخت دماسنج



تا اینجای کار، دنیای ما سیاه و سفید بود. یک کلید یا فشرده بود (۱) یا رها شده (۰). اما دنیای واقعی این طور نیست. دما ۲۵ درجه است، بعد می شود ۲۵.۱، بعد ۲۵.۲. نور کم و زیاد می شود. صدا موج دارد. به این مقادیر پیوسته **آنالوگ** می گویند. میکروکنترلر که یک چیپ دیجیتالی است، زبان آنالوگ را نمی فهمد. ما به یک مترجم نیاز داریم. این مترجم واحدی است به نام **ADC** (مبدل آنالوگ به دیجیتال) که درون ATmega32 تعبیه شده است. در این فصل، ما یک دماسنج دیجیتال می سازیم که دمای اتاق را اندازه گرفته و روی LCD (که در فصل ۷ یاد گرفتیم) نمایش دهد.

## مفهوم ADC: تبدیل ولتاژ به عدد

تصور کنید یک خط‌کش دارید که ۱۰۲۴ دندان دارد. واحد ADC ولتاژ ورودی (مثلاً ۰ تا ۵ ولت) را می‌گیرد و آن را با این خط‌کش اندازه می‌زند و به یک عدد بین ۰ تا ۱۰۲۳ تبدیل می‌کند.

- اگر ولتاژ ۰ باشد، عدد ADC می‌شود ۰.
- اگر ولتاژ ۵ باشد، عدد ADC می‌شود ۱۰۲۳.
- اگر ولتاژ ۲.۵ باشد، عدد ADC می‌شود ۵۱۲.

**دقت (Resolution):** چون ADC در میکروکنترلر ما ۱۰ بیتی است، دقت آن می‌شود یعنی ۱۰۲۴ پله.

## آشنایی با سنسور LM35

برای این پروژه از سنسور معروف LM35 استفاده می‌کنیم. این سنسور سه پایه دارد (VCC, GND, Output). ویژگی جذاب LM35 این است که خروجی آن به ازای هر ۱ درجه سانتی‌گراد، دقیقاً ۱۰ میلی‌ولت افزایش می‌یابد.

- دمای ۲۵ درجه -> خروجی ۲۵۰ میلی‌ولت (۰.۲۵ ولت)
- دمای ۱۰۰ درجه -> خروجی ۱۰۰۰ میلی‌ولت (۱ ولت)

## مرحله ۱: بستن مدار در پروتئوس

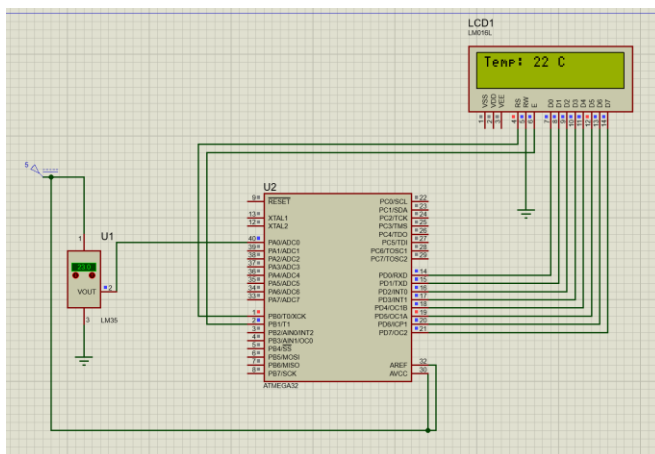
ما ترکیبی از مدار فصل قبل (LCD) و مدار جدید (LM35) را می‌خواهیم.

قطعات مورد نیاز:

- ATmega32
- LCD 16x2 (متصل به پورت D و B طبق فصل ۷)
- سنسور LM35 (در پروتئوس سرچ کنید LM35)

## اتصالات جدید:

۱. پایه VCC سنسور را به Power (5V) وصل کنید.
۲. پایه GND سنسور را به زمین وصل کنید.
۳. پایه وسط (Output) را به پایه PA0 میکروکنترلر (که کانال شماره ۰ مبدل ADC است) وصل کنید.
۴. نکته مهم: پایه AVCC و AREF میکروکنترلر را هم به ۵ ولت وصل کنید



## مرحله ۲: رجیسترهای ADC (اتاق فرمان)

برای راه اندازی ADC باید دو رجیستر اصلی را تنظیم کنیم:

### ۱. رجیستر ADMUX (انتخاب کانال و ولتاژ مرجع):

- بیت‌های REFS1, REFS0: تعیین می‌کنند ولتاژ مرجع (سقف خط‌کش) چقدر باشد. ما ولتاژ AVCC (۵ ولت) را انتخاب می‌کنیم.
- بیت‌های MUX0 تا MUX4: تعیین می‌کنند کدام پایه (PA0 تا PA7) خوانده شود.

### ۲. رجیستر ADCSRA (کنترل و وضعیت):

- ADEN: کلید اصلی روشن کردن ADC.

- **ADSC:** دستور شروع تبدیل (Start Conversion).
- **Prescaler:** سرعت ADC نباید خیلی زیاد باشد (بین ۵۰ تا ۲۰۰ کیلوهرتز). چون کلاک CPU ما ۱ مگاهرتز است، باید آن را تقسیم کنیم (مثلاً تقسیم بر ۸).

### مرحله ۳: کدنویسی دماسنج دیجیتال

در این کد، ما توابع LCD فصل قبل را خلاصه کرده‌ایم و توابع ADC را اضافه کرده‌ایم. از کتابخانه stdio.h برای تبدیل عدد به رشته استفاده می‌کنیم.

```

1. /*
2.  * main.c
3.  *
4.  * Description: Read Temp from LM35 on PA0 and display on LCD
5.  * Hardware: LM35 -> PA0 (ADC0), LCD -> PORTD & PORTB
6.  * Author: MahdiGhaemi
7.  */
8.
9. #define F_CPU 1000000UL
10. #include <avr/io.h>
11. #include <util/delay.h>
12. #include <stdio.h> // Required for sprintf function
13.
14. // --- LCD Configuration & Functions (Simplified from Chapter 7) ---
15. #define LCD_DATA PORTD
16. #define LCD_CTRL PORTB
17. #define RS_PIN 0
18. #define E_PIN 1
19.
20. void LCD_Cmd(unsigned char cmd) {
21.     LCD_DATA = cmd;
22.     LCD_CTRL &= ~(1<<RS_PIN); // Command Mode
23.     LCD_CTRL |= (1<<E_PIN); _delay_ms(1); LCD_CTRL &= ~(1<<E_PIN);
24.     _delay_ms(2);
25. }
26.
27. void LCD_Char(unsigned char data) {
28.     LCD_DATA = data;
29.     LCD_CTRL |= (1<<RS_PIN); // Data Mode
30.     LCD_CTRL |= (1<<E_PIN); _delay_ms(1); LCD_CTRL &= ~(1<<E_PIN);
31.     _delay_ms(2);
32. }
33.
34. void LCD_Init() {
35.     DDRD = 0xFF; DDRB = 0x03;
36.     _delay_ms(20);
37.     LCD_Cmd(0x38); LCD_Cmd(0x0C); LCD_Cmd(0x01); _delay_ms(2);
38. }

```



```

39.
40. void LCD_String(char *str) {
41.     int i;
42.     for(i=0; str[i]!=0; i++) LCD_Char(str[i]);
43. }
44.
45. // --- ADC Functions ---
46.
47. void ADC_Init() {
48.     // 1. Setup ADMUX Register
49.     // REFS0 = 1, REFS1 = 0 -> Use AVCC as Reference Voltage (5V)
50.     // MUX4..0 = 00000 -> Select Channel 0 (PA0) initially
51.     ADMUX = (1 << REFS0);
52.
53.     // 2. Setup ADCSRA Register
54.     // ADEN = 1 -> Enable ADC
55.     // ADPS1, ADPS0 = 1 -> Prescaler = 8 (1MHz / 8 = 125kHz which is
good)
56.     ADCSRA = (1 << ADEN) | (1 << ADPS1) | (1 << ADPS0);
57. }
58.
59. uint16_t ADC_Read(uint8_t channel) {
60.     // Select the channel (0-7) safely
61.     // First, clear the old channel bits (lower 5 bits) in ADMUX
62.     ADMUX &= 0xE0; // 0b11100000
63.     // Then set the new channel
64.     ADMUX |= channel;
65.
66.     // Start Conversion by setting ADSC bit to 1
67.     ADCSRA |= (1 << ADSC);
68.
69.     // Wait until conversion is complete
70.     // The ADSC bit will become 0 automatically when done
71.     while (ADCSRA & (1 << ADSC));
72.
73.     // Return the ADC value (0 to 1023)
74.     return ADCW; // ADCW is a special shortcut to read both ADCL and
ADCH
75. }
76.
77. // --- Main Function ---
78.
79. int main(void) {
80.     LCD_Init();    // Setup LCD
81.     ADC_Init();    // Setup ADC
82.
83.     uint16_t adc_value;
84.     float voltage;
85.     int temperature;
86.     char buffer[16]; // String buffer for display
87.
88.     LCD_String("Temp Monitor");

```

```

89.     _delay_ms(1000);
90.     LCD_Cmd(0x01); // Clear screen
91.
92.     while (1) {
93.         // 1. Read Analog value from Channel 0 (PA0)
94.         adc_value = ADC_Read(0);
95.
96.         /* 2. Math Calculations
97.            ADC Resolution = 1024 steps. Reference = 5V.
98.            Step Size = 5V / 1024 = 0.00488 V (approx 4.88mV)
99.            Voltage = ADC_Value * Step_Size
100.        */
101.        voltage = adc_value * (5.0 / 1024.0);
102.
103.        /* 3. Convert Voltage to Temperature
104.           LM35: 10mV per Degree -> 0.01V = 1 Degree
105.           Temp = Voltage / 0.01 OR Temp = Voltage * 100
106.        */
107.        temperature = (int)(voltage * 100);
108.
109.        // 4. Display on LCD
110.        // sprintf is like printf but prints into a string variable
111.        sprintf(buffer, "Temp: %d C ", temperature);
112.
113.        LCD_Cmd(0x80); // Move cursor to start of line 1
114.        LCD_String(buffer);
115.
116.        _delay_ms(500); // Update every half second
117.    }
118. }

```

## تحلیل نهایی و عمیق برنامه

۱. راز ریاضیات (The Math Behind): شاید بپرسید چرا  $\text{voltage} * 100$ ؟ بیایید

مهندسی فکر کنیم.

- پله‌های ADC: ولتاژ مرجع ما ۵ ولت است و ۱۰۲۴ پله داریم. پس هر پله برابر است با:

- حساسیت سنسور: سنسور LM35 به ازای هر درجه ۰.۰۱ ولت می‌دهد.

- بنابراین، اگر ADC عدد ۱ را بخواند، ولتاژ ۰.۰۰۴۸۸ است. برای تبدیل به دما:

یعنی هر یک واحد افزایش در عدد ADC، تقریباً نیم درجه دما را نشان می‌دهد.

۲. دستور `sprintf`: ما یک متغیر عددی (`temperature`) داریم و یک تابع `LCD_String` که فقط متن می‌گیرد. دستور `sprintf(buffer, "Temp: %d", temperature)` بسیار قدرتمند است. دقیقاً مثل `printf` در کامپیوتر عمل می‌کند، با این تفاوت که خروجی را روی مانیتور چاپ نمی‌کند، بلکه آن را تبدیل به متن کرده و در آرایه `buffer` می‌ریزد تا ما بتوانیم آن را به LCD بفرستیم.

۳. مفهوم **Polling** در تابع `ADC_Read`: خط `while (ADCSRA & (1 << ADSC))`; بسیار مهم است. عملیات تبدیل ولتاژ به عدد زمان‌بر است (چند میکروثانیه). CPU نمی‌تواند بلافاصله بعد از دستور شروع، عدد را بردارد. این حلقه `while` به CPU می‌گوید: "همینجا دور خودت بچرخ و صبر کن تا چراغ `ADSC` خاموش شود". وقتی خاموش شد یعنی تبدیل تمام شده و داده آماده است.

## فصل ۹: توقف ممنوع! جادوی وقفه‌ها (Interrupts)

در فصل‌های قبل، وقتی می‌خواستیم یک دکمه را بخوانیم، مجبور بودیم در حلقه `while(1)` مدام بپرسیم: «آیا دکمه فشرده شد؟... الان چی؟... الان چی؟». به این روش **Polling** (سرکشی مداوم) می‌گویند. مشکل بزرگ اینجاست که وقتی CPU مشغول این سوال تکراری است، یا مشغول یک `delay` طولانی است، نمی‌تواند کار دیگری انجام دهد. اگر یک اتفاق اورژانسی بیفتد، میکروکنترلر شاید اصلاً متوجه نشود!

**راه حل: وقفه (Interrupt)** وقفه دقیقاً مثل زنگ در خانه است. شما مشغول دیدن فیلم هستید (برنامه اصلی) و اصلاً به در نگاه نمی‌کنید. وقتی پستیچی زنگ را می‌زند (سیگنال سخت‌افزاری)، شما فیلم را **Pause** می‌کنید، می‌روید بسته را می‌گیرید (اجرای تابع وقفه)، و دوباره برمی‌گردید و ادامه فیلم را می‌بینید.

### سناریوی پروژه

ما می‌خواهیم قدرت واقعی وقفه را ببینیم:

۱. **وظیفه اصلی: (Main Loop)** یک LED سبز دائماً هر ۱ ثانیه روشن و خاموش می‌شود. این LED نماد کارهای روزمره میکروکنترلر است.
۲. **وظیفه اورژانسی: (Interrupt)** یک دکمه داریم که به محض فشرده شدن، یک LED قرمز را روشن/خاموش (Toggle) می‌کند.

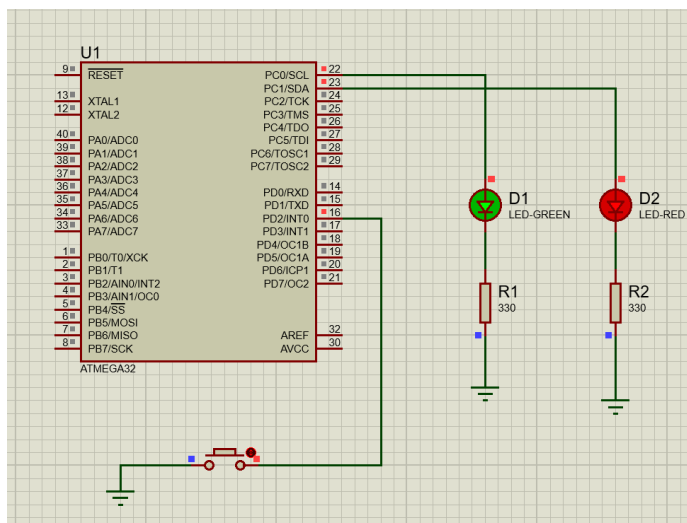
نکته طلایی: حتی اگر میکروکنترلر وسط آن ۱ ثانیه تاخیر (Delay) باشد و خوابیده باشد، با زدن دکمه باید بیدار شود و LED قرمز را عوض کند!

## سخت‌افزار: پایه‌های خاص

در ATmega32 ما نمی‌توانیم دکمه وقفه را به هر پایه‌ای وصل کنیم. ما پایه‌های مخصوصی به نام **INT0** و **INT1** و **INT2** داریم. برای این پروژه از **INT0** استفاده می‌کنیم که روی پایه **PD2** (پایه شماره ۱۶) قرار دارد.

اتصالات در پروتئوس:

۱. LED سبز: (Main) به پایه **PC0** وصل شود.
  ۲. LED قرمز: (Interrupt) به پایه **PC1** وصل شود.
  ۳. دکمه: (Button) یک سر به **PD2** و سر دیگر به **GND** زمین.
- تنظیمات رجیسترها (اتاق فرمان وقفه)



برای فعال‌سازی **INT0** باید سه مرحله را طی کنیم:

۱. اجازه دادن (**GICR**): باید به میکرو بگوییم "لطفا وقفه شماره ۰ را فعال کن". بیت **INT0** را یک می‌کنیم.

۲. حساسیت: (MCUCR) باید بگوییم کی تحریک شود؟ وقتی ولتاژ ۵ ولت است؟ یا وقتی دارد از ۵ به ۰ می‌رسد (Falling Edge)؟ ما "لبه پایین‌رونده" یا Falling Edge را انتخاب می‌کنیم چون دکمه ما به زمین وصل است.
۳. کلید اصلی: (SREG) یک فیوز اصلی وجود دارد به نام "وقفه سراسری Global Interrupt" تا این فیوز را نزنیم، هیچ وقفه‌ای کار نمی‌کند.

### کدنویسی پروژه وقفه

در کد زیر، ما یک تابع عجیب به نام ISR می‌بینیم. این تابع توسط ما در برنامه صدا زده نمی‌شود! بلکه وقتی دکمه زده شد، خود سخت‌افزار میکروکنترلر به صورت خودکار به اینجا می‌پرد.

```

1. /*
2.  * main.c
3.  *
4.  * Description: Blinking LED in main loop, Toggling another LED via INT0
interrupt
5.  * Hardware:
6.  * - Green LED -> PC0 (Main Task)
7.  * - Red LED   -> PC1 (Interrupt Task)
8.  * - Button    -> PD2 (INT0 Pin)
9.  * Author: MahdiGhaemi
10. */
11.
12. #define F_CPU 1000000UL
13. #include <avr/io.h>
14. #include <util/delay.h>
15. #include <avr/interrupt.h> // Essential header for ISR
16.
17. // --- Interrupt Service Routine (ISR) for INT0 ---
18. // This code runs AUTOMATICALLY when the button is pressed
19. ISR(INT0_vect)
20. {
21.     // Toggle the Red LED (PC1)
22.     // XOR operation (^) flips the bit
23.     PORTC ^= (1 << 1);
24. }
25.
26. int main(void)

```

```

27. {
28.     // --- Configuration ---
29.
30.     // 1. Setup LEDs
31.     // Set PC0 and PC1 as Output
32.     DDRC |= (1 << 0) | (1 << 1);
33.
34.     // 2. Setup Button (INT0 pin is PD2)
35.     // Set PD2 as Input
36.     DDRD &= ~(1 << 2);
37.     // Enable Internal Pull-up Resistor for PD2
38.     PORTD |= (1 << 2);
39.
40.     // 3. Configure External Interrupt 0 (INT0)
41.
42.     // Step A: Trigger on Falling Edge (When button is pressed)
43.     // Look at Datasheet: ISC01=1, ISC00=0 in MCUCR register means Falling
Edge
44.     MCUCR |= (1 << ISC01);
45.     MCUCR &= ~(1 << ISC00); // Usually 0 by default, but good to be safe
46.
47.     // Step B: Enable INT0 Request
48.     // Set INT0 bit in General Interrupt Control Register (GICR)
49.     GICR |= (1 << INT0);
50.
51.     // Step C: Enable Global Interrupts
52.     // This is the "Master Switch"
53.     sei();
54.
55.     // --- Main Loop ---
56.     while (1)
57.     {
58.         // Task A: Just blink the Green LED slowly
59.         // This simulates a busy CPU
60.
61.         PORTC ^= (1 << 0); // Toggle Green LED
62.         _delay_ms(1000);    // Wait for 1 second
63.
64.         // Notice: We do NOT check the button here!
65.         // The hardware watches PD2 for us.
66.     }
67. }

```

## تحلیل و بررسی دقیق

بیا یاد اتفاقی که می‌افتد را صحنه آهسته کنیم:

۱. اجرای عادی: برنامه داخل (1)while است LED. سبز روشن می‌شود و CPU وارد دستور delay\_ms(1000) می‌شود. در این لحظه CPU عملاً خواب است و دارد ثانیه‌ها را می‌شمارد.
  ۲. وقوع حادثه: شما دکمه را فشار می‌دهید. ولتاژ پایه PD2 از ۵ ولت به ۰ ولت می‌رسد (Falling Edge).
  ۳. واکنش سخت‌افزاری: واحد وقفه میکروکنترلر بیدار می‌شود و می‌بیند که شرط Falling Edge رخ داده است.
  ۴. تله‌پورت: (Context Switch) میکروکنترلر بلافاصله هر کاری که در delay انجام می‌داد را متوقف می‌کند (جای خط کد را در حافظه ذخیره می‌کند) و ناگهان به داخل تابع ISR(INT0\_vect) می‌پرد.
  ۵. اجرای وقفه: دستور  $PORTC \wedge = (1 << 1)$  اجرا می‌شود و LED قرمز تغییر وضعیت می‌دهد. این کار شاید چند میکروثانیه طول بکشد.
  ۶. بازگشت: به محض تمام شدن تابع ISR، میکروکنترلر به همان جایی از delay برمی‌گردد که قطع شده بود و ادامه آن ۱ ثانیه را طی می‌کند.
- نتیجه: شما احساس می‌کنید دو برنامه همزمان (Parallel) اجرا می‌شوند، در حالی که میکروکنترلر فقط خیلی سریع بین کارها جابجا می‌شود.



## فصل ۱۰: ارباب زمان؛ تایمرها و کانترها (Timers)

در فصل قبل دیدیم که delay دشمن کارایی است، چون پردازنده را فلج می‌کند. در فصل اینترنت هم یاد گرفتیم چطور به دکمه‌ها واکنش سریع نشان دهیم. اما اگر بخواهیم کاری را "سر زمان مشخص" انجام دهیم چه؟ مثلاً ساختن یک ساعت دیجیتال دقیق، یا چشمک زدن یک LED دقیقاً هر ۵۰۰ میلی‌ثانیه، بدون اینکه CPU را متوقف کنیم.

اینجاست که تایمرها (Timers) وارد می‌شوند. تایمرها مدارهای مستقلی درون میکروکنترلر هستند که مثل یک کرنومتر جداگانه، پالس‌های ساعت را می‌شمارند و وقتی به عدد خاصی رسیدند، به CPU خبر می‌دهند (اینترنت می‌زنند).

### سناریوی پروژه: مولتی‌تسکینگ واقعی

ما می‌خواهیم دو کار همزمان انجام دهیم که با delay غیرممکن است:

۱. تایمر ۱ (۱۶ بیتی): وظیفه دارد دقیقاً هر ۱ ثانیه یک بار، شمارنده روی LCD را یکی زیاد کند (ساعت دقیق).
۲. تایمر ۰ (۸ بیتی): وظیفه دارد هر ۵۰۰ میلی ثانیه (تقریبی) یک LED را خاموش و روشن کند.

### مفهوم Prescaler تقسیم کننده فرکانس

سرعت کلاک میکروکنترلر ما ۱ مگاهرتز است (۱ میلیون تیک در ثانیه). تایمرها نمی توانند تا ۱ میلیون بشمارند (تایمر ۸ بیتی تا ۲۵۵ و ۱۶ بیتی تا ۶۵۵۳۵ می شمارد). برای حل این مشکل از Prescaler استفاده می کنیم. این قطعه سرعت شمارش را کم می کند. مثل عقربه ثانیه شمار که ۶۰ برابر کندتر از میلی ثانیه می چرخد. ما سرعت ۱ مگاهرتز را بر ۶۴ تقسیم می کنیم.

$$1,000,000 \div 64 = 15,625$$

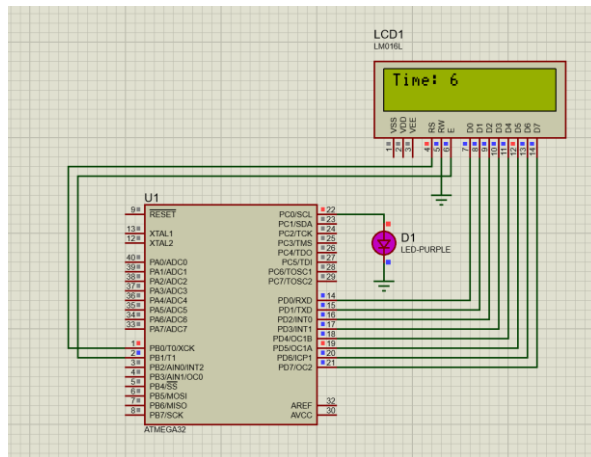
یعنی تایمر ما در هر ثانیه ۱۵۶۲۵ بار می شمارد. پس اگر به تایمر بگوییم "تا ۱۵۶۲۵ بشمار"، دقیقاً ۱ ثانیه طول می کشد.

### سخت افزار مورد نیاز

۱. LCD 16x2. متصل به پورت D و B طبق فصل ۷.

۲. LED. متصل به پایه PC0.

۳. میکروکنترلر ATmega32.



## کدنویسی: رقص هماهنگ تایمرها

در این کد از تایمر ۱ در مد CTC استفاده می‌کنیم (بهترین مد برای زمان‌سنجی دقیق). در این مد، تایمر تا عددی که ما می‌گوییم می‌شمارد، سپس صفر می‌شود و اینترپت می‌زند. برای تایمر ۰ از مد Overflow استفاده می‌کنیم (ساده‌ترین مد) تا فقط LED را کنترل کند. نکته مهم: متغیرهایی که در اینترپت تغییر می‌کنند باید از نوع volatile باشند تا کامپایلر آن‌ها را به اشتباه حذف نکند.

```
1. /*
2.  * main.c
3.  *
4.  * Description:
5.  * 1. Timer1 (16-bit) counts 1 second precisely for LCD Clock.
6.  * 2. Timer0 (8-bit) blinks LED every ~500ms.
7.  * Author: MahdiGhaemi
8.  */
9.
10. #define F_CPU 1000000UL
11. #include <avr/io.h>
12. #include <util/delay.h>
13. #include <avr/interrupt.h>
14. #include <stdio.h> // For sprintf
15.
16. // --- Global Variables ---
17. // 'volatile' tells compiler this variable can change unexpectedly (by ISR)
18. volatile int seconds_counter = 0;
19. volatile int timer0_overflows = 0;
20.
21. // --- LCD Functions (Simplified for brevity - assumes Chapter 7 code) ---
```

```

22. #define LCD_DATA PORTD
23. #define LCD_CTRL PORTB
24. #define RS 0
25. #define EN 1
26.
27. void LCD_Cmd(unsigned char cmd);
28. void LCD_Char(unsigned char data);
29. void LCD_Init(void);
30. void LCD_String(char *str);
31. void LCD_SetCursor(uint8_t r, uint8_t c);
32.
33. // --- Timer 1 Configuration (The Precise 1-Second Clock) ---
34. void Timer1_Init()
35. {
36.     // Mode 4: CTC (Clear Timer on Compare Match)
37.     // We want to count up to a specific value, then reset automatically.
38.     // WGM12 = 1 sets CTC mode.
39.     TCCR1B |= (1 << WGM12);
40.
41.     // Set Prescaler to 64
42.     // CS11 = 1, CS10 = 1
43.     TCCR1B |= (1 << CS11) | (1 << CS10);
44.
45.     // Calculate Compare Value (OCR1A)
46.     // Target = 1 second. F_CPU = 1MHz. Prescaler = 64.
47.     // Ticks = 1,000,000 / 64 = 15625
48.     // Since counting starts from 0, we subtract 1: 15624
49.     OCR1A = 15624;
50.
51.     // Enable Timer1 Compare Match A Interrupt
52.     TIMSK |= (1 << OCIE1A);
53. }
54.
55. // --- Timer 0 Configuration (The Blinking LED) ---
56. void Timer0_Init()
57. {
58.     // Normal Mode (Overflow Mode) is default (WGM00=0, WGM01=0)
59.
60.     // Set Prescaler to 1024 (Maximum slowing down)
61.     // CS02=1, CS00=1
62.     TCCR0 |= (1 << CS02) | (1 << CS00);
63.
64.     // Enable Timer0 Overflow Interrupt
65.     TIMSK |= (1 << TOIE0);
66. }
67.
68. // --- ISR for Timer 1 (Runs every 1 second) ---
69. ISR(TIMER1_COMPA_vect)
70. {
71.     // Increment the clock counter
72.     seconds_counter++;
73. }
74.
75. // --- ISR for Timer 0 (Runs every time Timer0 overflows) ---
76. // With 1024 prescaler, overflow happens every ~262ms

```

```

77. ISR(TIMER0_OVF_vect)
78. {
79.     timer0_overflows++;
80.
81.     // We want 500ms.
82.     // 2 overflows * 262ms = 524ms (Close enough for blinking)
83.     if (timer0_overflows >= 2)
84.     {
85.         // Toggle LED on PC0
86.         PORTC ^= (1 << 0);
87.
88.         // Reset overflow counter
89.         timer0_overflows = 0;
90.     }
91. }
92.
93. int main(void)
94. {
95.     // Setup Hardware
96.     DDRC |= (1 << 0); // PC0 as Output (LED)
97.
98.     // Initialize Modules
99.     LCD_Init();
100.    LCD_String("Time: 0");
101.
102.    Timer1_Init();
103.    Timer0_Init();
104.
105.    // Enable Global Interrupts (The Master Switch)
106.    sei();
107.
108.    char buffer[16];
109.    int last_displayed_second = -1;
110.
111.    while (1)
112.    {
113.        // Only update LCD if the second has changed
114.        // This is much more efficient than printing constantly
115.        if (seconds_counter != last_displayed_second)
116.        {
117.            sprintf(buffer, "Time: %d  ", seconds_counter);
118.            LCD_Cmd(0x80); // Move cursor to start
119.            LCD_String(buffer);
120.
121.            // Update the tracker
122.            last_displayed_second = seconds_counter;
123.        }
124.
125.        // CPU is free here! It can do other calculations
126.        // while LED blinks and Clock ticks automatically.
127.    }
128. }
129.
130. // --- LCD Implementation (Paste below main or in separate file) ---
131. void LCD_Cmd(unsigned char cmd) {

```

```

132.     LCD_DATA = cmd; LCD_CTRL &= ~(1<<RS);
133.     LCD_CTRL |= (1<<EN); _delay_ms(1); LCD_CTRL &= ~(1<<EN); _delay_ms(2);
134. }
135. void LCD_Char(unsigned char data) {
136.     LCD_DATA = data; LCD_CTRL |= (1<<RS);
137.     LCD_CTRL |= (1<<EN); _delay_ms(1); LCD_CTRL &= ~(1<<EN); _delay_ms(2);
138. }
139. void LCD_Init() {
140.     DDRD = 0xFF; DDRB |= (1<<RS)|(1<<EN);
141.     _delay_ms(20); LCD_Cmd(0x38); LCD_Cmd(0x0C); LCD_Cmd(0x01);
    _delay_ms(2);
142. }
143. void LCD_String(char *str) {
144.     while(*str) LCD_Char(*str++);
145. }

```

## تحلیل نهایی

۱. چرا مد CTC برای ساعت بهتر است؟

در مد CTC (تایمر ۱)، ما سقف شمارش را روی ۱۵۶۲۴ قفل کردیم. تایمر دقیقاً به این عدد که می‌رسد، خودش اتوماتیک صفر می‌شود و دوباره می‌شمارد. این یعنی خطای زمانی ما صفر است و ساعت ما حتی بعد از گذشت چند روز هم دقیق کار می‌کند.

۲. چرا مد Overflow برای LED استفاده شد؟

برای چشمک زدن LED دقت میلی‌ثانیه‌ای خیلی مهم نیست. تایمر ۰ یک تایمر ۸ بیتی ساده است. ما آن را روی کمترین سرعت (Prescaler 1024) گذاشتیم. در این حالت هر ۲۶۲ میلی‌ثانیه یک بار "سرریز" می‌کند (از ۲۵۵ به ۰ می‌پرد). ما در وقفه می‌گوییم: "هر وقت ۲ بار سرریز کردی (یعنی حدود ۵۲۴ میلی‌ثانیه شد)، لامپ را عوض کن". این روش ساده‌ترین راه برای زمان‌سنجی‌های غیر دقیق است.

۳. متغیر: volatile

دقت کنید متغیر seconds\_counter در بالای برنامه با کلمه کلیدی volatile تعریف شد.

چرا؟

چون این متغیر در تابع main تغییر نمی‌کند، بلکه در تابع ISR (که ناگهانی اجرا می‌شود) تغییر می‌کند. اگر volatile نگذاریم، کامپایلر فکر می‌کند این متغیر بی‌استفاده است و ممکن است آن را حذف کند یا آپدیت نکند.

# فصل ۱۱: پروژه نهایی؛ ساخت منوی هوشمند با کپد و LCD

در این پروژه ما یک "سیستم چند منظوره" می‌سازیم. میکروکنترلر منتظر می‌ماند تا شما یک کلید را فشار دهید.

- اگر کلید '1' را بزنید: یک رقص نور باینری روی LED ها اجرا می‌شود.
- اگر کلید '2' را بزنید: دستگاه شروع به محاسبه اعداد اول (Prime Numbers) می‌کند و آن‌ها را هم روی LCD و هم به صورت باینری روی LED ها نمایش می‌دهد.

## سخت‌افزار پروژه (مدار)

این پروژه از ۳ پورت میکروکنترلر استفاده می‌کند:

۱. پورت D برای LCD در مد ۴ بیتی:

- پایه PD0 به RS نمایشگر.
- پایه PD1 به E نمایشگر.
- پایه RW نمایشگر به زمین (GND).
- پایه‌های PD4 تا PD7 به پایه‌های D4 تا D7 نمایشگر وصل می‌شوند. پایه‌های D0 تا D3 نمایشگر آزاد می‌مانند.

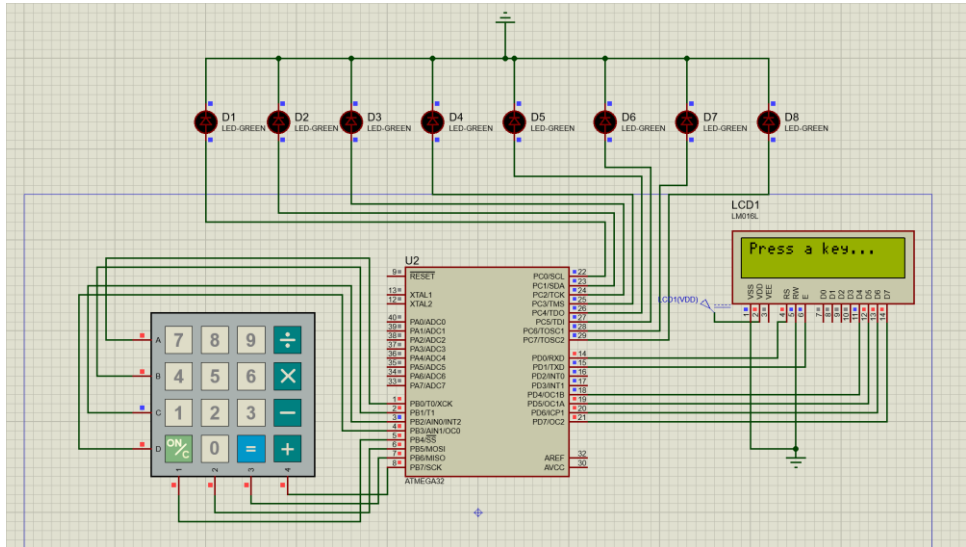
۲. پورت B برای کپد ۴ در ۴:

- پایه‌های PB0 تا PB3 به سطرهای کپد (Rows).
- پایه‌های PB4 تا PB7 به ستون‌های کپد (Cols).



### ۳. پورت C برای LED ها:

- ۸ عدد LED به پایه‌های PC0 تا PC7 وصل می‌شوند (برای نمایش خروجی).



### کدنویسی پروژه (سیستم منو)

این کد شامل توابع پایین برای کنترل سخت‌افزار و توابع سطح بالا برای منطق برنامه‌هاست. کامنت‌های کد به زبان انگلیسی نوشته شده‌اند تا استاندارد باشند.

```
1. /*
2.  * Final_Project_Menu.c
3.  *
4.  * Description: A multi-function system with Keypad Menu and 4-bit LCD.
5.  * Programs:
6.  * 1. Binary LED Counter / Shifter
7.  * 2. Prime Number Calculator
8.  *
9.  * Hardware Connections:
10. * - LCD Data (4-bit): PD4-PD7 -> LCD D4-D7
11. * - LCD Control:      PD0 -> RS, PD1 -> E
12. * - Keypad:           PORTB (Rows: PB0-3, Cols: PB4-7)
13. * - LEDs:             PORTC (All 8 pins)
14. *
15. * Author: Student
16. */
17.
18. #define F_CPU 1000000UL
19. #include <avr/io.h>
```

```

20. #include <util/delay.h>
21.
22. // --- LCD Configurations (4-Bit Mode) ---
23. #define RS PD0
24. #define E PD1
25. #define LCD_PORT PORTD
26.
27. // Function to send a command to the LCD (e.g., Clear, Home)
28. void lcd_command(unsigned char cmd) {
29.     // Send Upper Nibble (Most Significant 4 bits)
30.     LCD_PORT = (cmd & 0xF0); // Mask lower 4 bits
31.     LCD_PORT &= ~(1 << RS); // RS = 0 for Command
32.     LCD_PORT |= (1 << E); // Enable Pulse High
33.     _delay_ms(1);
34.     LCD_PORT &= ~(1 << E); // Enable Pulse Low
35.     _delay_ms(2);
36.
37.     // Send Lower Nibble (Least Significant 4 bits)
38.     LCD_PORT = ((cmd << 4) & 0xF0); // Shift lower bits to upper position
39.     LCD_PORT |= (1 << E); // Enable Pulse High
40.     _delay_ms(1);
41.     LCD_PORT &= ~(1 << E); // Enable Pulse Low
42.     _delay_ms(2);
43. }
44.
45. // Function to send data (characters) to the LCD
46. void lcd_data(unsigned char data) {
47.     // Send Upper Nibble
48.     LCD_PORT = (data & 0xF0);
49.     LCD_PORT |= (1 << RS); // RS = 1 for Data
50.     LCD_PORT |= (1 << E);
51.     _delay_ms(1);
52.     LCD_PORT &= ~(1 << E);
53.     _delay_ms(2);
54.
55.     // Send Lower Nibble
56.     LCD_PORT = ((data << 4) & 0xF0);
57.     LCD_PORT |= (1 << RS); // RS = 1 for Data
58.     LCD_PORT |= (1 << E);
59.     _delay_ms(1);
60.     LCD_PORT &= ~(1 << E);
61.     _delay_ms(2);
62. }
63.
64. // Initialize LCD in 4-bit mode
65. void lcd_init() {
66.     DDRD = 0xFF; // Set LCD Port as Output
67.     _delay_ms(15);
68.
69.     // Initialization sequence for 4-bit mode
70.     lcd_command(0x02); // Initialize in 4-bit mode
71.     lcd_command(0x28); // 2 lines, 5x7 matrix
72.     lcd_command(0x0C); // Display ON, Cursor OFF
73.     lcd_command(0x06); // Increment cursor (shift right)
74.     lcd_command(0x01); // Clear display screen

```

```

75.     _delay_ms(2);
76. }
77.
78. // Helper function to clear LCD
79. void lcd_clear() {
80.     lcd_command(0x01);
81.     _delay_ms(2);
82. }
83.
84. // Function to print a string on LCD
85. void lcd_print(char *str) {
86.     while (*str) {
87.         lcd_data(*str++);
88.     }
89. }
90.
91. // Helper to print a byte as binary string (e.g., "0001111")
92. void lcd_print_binary(unsigned char val) {
93.     for (int i = 7; i >= 0; i--) {
94.         if (val & (1 << i))
95.             lcd_data('1');
96.         else
97.             lcd_data('0');
98.     }
99. }
100.
101. // --- Keypad Configurations ---
102. // Define the Keypad Layout
103. char keypad[4][4] = {
104.     {'7','8','9','/'},
105.     {'4','5','6','*'},
106.     {'1','2','3','-'},
107.     {'0','0','=','+'}
108. };
109.
110. void keypad_init() {
111.     // PB0-PB3 (Rows) as Output, PB4-PB7 (Cols) as Input
112.     DDRB = 0x0F;
113.     // Enable Pull-up resistors for Inputs (Cols) and set Rows High
114.     PORTB = 0xF0;
115. }
116.
117. // Function to scan the keypad and return the pressed key
118. char keypad_getkey() {
119.     for (uint8_t row = 0; row < 4; row++) {
120.         PORTB = 0xFF; // Reset Port
121.         PORTB &= ~(1 << row); // Ground one row at a time (Active Low)
122.         _delay_ms(1);
123.
124.         // Check columns
125.         for (uint8_t col = 0; col < 4; col++) {
126.             // Check if any column pin is Low (Pressed)
127.             // Columns are connected to PB4, PB5, PB6, PB7
128.             if (!(PINB & (1 << (col + 4)))) {
129.                 _delay_ms(50); // Debounce delay

```

```

130.             while (!(PINB & (1 << (col + 4)))); // Wait for key release
131.             return keypad[row][col]; // Return the character
132.         }
133.     }
134. }
135. return 0; // No key pressed
136. }
137.
138. // --- Sub-Program 1: LED Shifter ---
139. void program1() {
140.     DDRC = 0xFF; // Set LEDs as Output
141.     unsigned char pattern = 0x01;
142.
143.     lcd_clear();
144.     lcd_print("Program 1 Running");
145.     _delay_ms(1000);
146.
147.     // Run the loop for a specific duration or count
148.     for (long i = 0; i < 20; i++) {
149.         PORTC = pattern;
150.         pattern <<= 1; // Shift left
151.         if (pattern == 0)
152.             pattern = 0x01; // Reset pattern
153.
154.         lcd_clear();
155.         lcd_print("LEDs:");
156.         lcd_command(0xC0); // Move to second line
157.         lcd_print_binary(PORTC);
158.         _delay_ms(300);
159.     }
160.     lcd_clear();
161.     lcd_print("Program 1 Done");
162.     _delay_ms(2000);
163. }
164.
165. // --- Sub-Program 2: Prime Number Finder ---
166. // Function to check if a number is prime
167. uint8_t is_prime(uint16_t n) {
168.     if (n < 2) return 0;
169.     for (uint16_t i = 2; i * i <= n; i++) {
170.         if (n % i == 0)
171.             return 0;
172.     }
173.     return 1;
174. }
175.
176. void program2() {
177.     DDRC = 0xFF; // Set LEDs as Output
178.
179.     lcd_clear();
180.     lcd_print("Program 2 Running");
181.     _delay_ms(1000);
182.
183.     // Check numbers up to 512
184.     for (uint16_t num = 2; num <= 255; num++) {

```

```

185.         if (is_prime(num)) {
186.             PORTC = (unsigned char)num; // Show binary on LEDs
187.             lcd_clear();
188.             lcd_print("Prime: ");
189.
190.             // Convert integer to string manually for display
191.             char buffer[6];
192.             uint16_t temp = num;
193.             int i = 0;
194.             if (temp == 0) buffer[i++] = '0';
195.             while (temp > 0 && i < 5) {
196.                 buffer[i++] = (temp % 10) + '0';
197.                 temp /= 10;
198.             }
199.             // Print buffer in reverse order
200.             while (i > 0) lcd_data(buffer[--i]);
201.
202.             lcd_command(0xC0); // Second line
203.             lcd_print_binary(PORTC); // Show binary representation
204.             _delay_ms(500);
205.         }
206.     }
207.     lcd_clear();
208.     lcd_print("Program 2 Done");
209.     _delay_ms(2000);
210. }
211.
212. // --- Main Loop ---
213. int main(void) {
214.     DDRD = 0xFF; // Set LCD Port as Output
215.     keypad_init();
216.     lcd_init();
217.
218.     lcd_print("Press a key...");
219.
220.     while (1) {
221.         char key = keypad_getkey(); // Scan for key press
222.
223.         if (key) {
224.             lcd_clear();
225.             lcd_print("You pressed:");
226.             lcd_command(0xC0); // Second Line
227.             lcd_data(key);
228.             _delay_ms(1000);
229.
230.             // Menu Logic
231.             if (key == '1') {
232.                 program1(); // Run LED Shifter
233.                 lcd_clear();
234.                 lcd_print("Press a key...");
235.             }
236.             else if (key == '2') {
237.                 program2(); // Run Prime Calculator
238.                 lcd_clear();
239.                 lcd_print("Press a key...");

```

```

240.         }
241.         else {
242.             lcd_clear();
243.             lcd_print("Invalid Option");
244.             _delay_ms(1000);
245.             lcd_clear();
246.             lcd_print("Press 1 or 2");
247.         }
248.     }
249. }
250. return 0;
251. }

```

## تحلیل فنی پروژه

در این پروژه ما با محدودیت پین‌ها مواجه بودیم، بنابراین مجبور شدیم از تکنیک‌های فشرده‌سازی استفاده کنیم. بیایید ۳ بخش اصلی این کد را زیر ذره‌بین ببریم:

### ۱. تکنیک ارسال ۴ بیتی به LCD یا همان صرفه‌جویی در سیم

ال‌سی‌دی‌ها معمولاً ۸ پایه دیتا یعنی D0 تا D7 دارند. اگر بخواهیم همه را وصل کنیم، یک پورت کامل میکروکنترلر اشغال می‌شود. راه حل چیست؟ استفاده از مد ۴ بیتی. در این روش، ما فقط پایه‌های D4 تا D7 را وصل می‌کنیم. اما چطور ۸ بیت داده را با ۴ سیم بفرستیم؟ اطلاعات را دو نیمه می‌کنیم:

- نیمه اول یا **High Nibble**: شامل ۴ بیت پرارزش.
- نیمه دوم یا **Low Nibble**: شامل ۴ بیت کم‌ارزش.

در کد `lcd_data` چه اتفاقی می‌افتد؟ فرض کنید می‌خواهیم حرف A با کد باینری ۰۱۰۰۰۰۰۱ را بفرستیم:

۱. ابتدا ۰۱۰۰ یعنی بخش اول را روی پین‌ها می‌گذاریم و یک پالس به پایه E می‌دهیم. LCD این نصفه را می‌گیرد و در حافظه موقت نگه می‌دارد.

۲. سپس ۰۰۰۱ یعنی بخش دوم را روی همان پین‌ها می‌گذاریم و دوباره پالس E می‌دهیم.

۳. ال‌سی‌دی خودش این دو تکه را به هم می‌چسباند و می‌فهمد که حرف A بوده است.

## ۲. الگوریتم جاروب کردن کیپد یا Keypad Scanning

این پیچیده‌ترین و جذاب‌ترین بخش پروژه است. کیپد ماتریسی ۱۶ کلید دارد، اما فقط ۸ پایه یعنی ۴ سطر و ۴ ستون دارد. داخل کیپد، کلیدها در محل تقاطع سطرها و ستون‌ها قرار دارند. وقتی کلیدی فشرده می‌شود، سیم سطر مربوطه به سیم ستون مربوطه اتصال کوتاه می‌شود. استراتژی ما یا الگوریتم اسکن: ما میکروکنترلر را طوری برنامه ریزی می‌کنیم که مدام از سطرها سوال بپرسد!

### • تنظیمات سخت‌افزاری:

○ **سطرها یا Rows** را خروجی تعریف کردیم یعنی DDRB بیت‌های پایین ۱ شدند.

○ **ستون‌ها یا Cols** را ورودی تعریف کردیم یعنی DDRB بیت‌های بالا ۰ شدند و مقاومت‌های پول-آپ یا Pull-up داخلی را روشن کردیم یعنی PORTB بیت‌های بالا ۱ شدند.

## مراحل اسکن در تابع: keypad\_getkey

۱. حالت پیش‌فرض: تمام ستون‌ها به دلیل پول-آپ، ولتاژ ۵ ولت یا منطق ۱ دارند.
۲. شروع پرسش یا حلقه: **Row** میکروکنترلر سطر اول را صفر یا GND می‌کند و بقیه سطرها را ولتاژ بالا نگه می‌دارد. الان فقط سطر اول فعال است.
۳. چک کردن جواب یا حلقه: **Col** میکروکنترلر نگاه می‌کند ببیند آیا ولتاژ هیچکدام از ستون‌ها "صفر" شده است؟

○ اگر دکمه‌ای در سطر اول فشرده شده باشد، ولتاژ صفرِ سطر به ستون منتقل می‌شود و میکرو می‌فهمد: «آها! ستون ۲ صفر شد، پس دکمه‌ای که در تقاطع سطر ۱ و ستون ۲ است فشرده شده».

۴. **تکرار:** اگر دکمه‌ای در سطر اول نبود، میکروکنترلر سطر اول را غیرفعال یعنی ۱ می‌کند و سطر دوم را صفر یا GND می‌کند و دوباره ستون‌ها را چک می‌کند. این فرآیند آنقدر سریع انجام می‌شود یعنی چند هزار بار در ثانیه که شما فکر می‌کنید تمام کلیدها همزمان فعال هستند.

**رفع نویز یا Debouncing:** کلیدهای مکانیکی وقتی فشرده می‌شوند، برای چند میلی‌ثانیه می‌لرزند و قطع و وصل می‌شوند. دستور `delay_ms` با مقدار ۵۰ برای این است که صبر کنیم تا لرزش کلید تمام شود و اتصال پایدار گردد.

### ۳. منطق برنامه‌های فرعی

**برنامه ۱: رقص نور شیفت یا Shifter** در این برنامه از عملگر شیفت به چپ استفاده کردیم. دستور `pattern <<= 1` یعنی بیت روشن را یک خانه به چپ هل بده.

- مرحله ۱ 00000001: یعنی لامپ اول روشن
- مرحله ۲ 00000010: یعنی لامپ دوم روشن
- ...وقتی به انتها رسید یعنی عدد صفر شد، دوباره آن را ۰x01 می‌کنیم تا چرخه تکرار شود.

**برنامه ۲: تشخیص عدد اول یا Prime Number** عدد اول عددی است که فقط بر خودش و ۱ بخش‌پذیر باشد. تابع `is_prime` یک حلقه دارد که از ۲ شروع می‌شود و تا جذر آن عدد بالا می‌رود. اگر عدد ما مثلاً  $n$  بر هر کدام از این شمارنده‌ها یعنی  $i$  بخش‌پذیر باشد و باقی‌مانده تقسیم صفر شود، یعنی عدد اول نیست و تابع عدد ۰ برمی‌گرداند. اگر حلقه تمام شد و هیچ مقسوم‌علیه‌ی پیدا نشد، عدد اول است.



**نمایش عدد روی LCD:** چون تابع `sprintf` حجم زیادی از حافظه را می‌گیرد، ما در این بخش از روش تبدیل دستی استفاده کردیم:

۱. باقی‌مانده عدد بر ۱۰ را می‌گیریم که رقم یکان بدست می‌آید.
۲. عدد را بر ۱۰ تقسیم می‌کنیم که یکان حذف می‌شود.
۳. این کار را تکرار می‌کنیم تا تمام ارقام جدا شوند و آن‌ها را روی LCD چاپ می‌کنیم.

**نتیجه‌گیری نهایی:** این پروژه عصاره تمام چیزهایی بود که یاد گرفتیم:

- مدیریت پورت‌ها مانند DDR و PORT و PIN
- زمان‌بندی یا Delay
- کار با LCD یا نمایشگر
- الگوریتم‌نویسی مانند حلقه‌های تو در تو برای کیپد و ریاضیات

## فصل ۱۲: دمیدن روح در کالبد؛ پروگرام کردن و نکات اجرایی

تا این لحظه، تمام کدهای ما فقط در دنیای شبیه‌سازی (پروتئوس) زنده بودند. اما لذت واقعی الکترونیک زمانی است که کد را روی یک چیپ واقعی بریزید، آن را لمس کنید و عملکردش را ببینید.

در این فصل یاد می‌گیریم چگونه فایل ساخته شده در کامپیوتر را به حافظه میکروکنترلر منتقل کنیم.

### بخش اول: معرفی ابزار انتقال (پروگرامر)

برای انتقال فایل hex از کامپیوتر به میکروکنترلر، به یک واسط سخت‌افزاری به نام پروگرامر (Programmer) نیاز داریم. در اینجا دو مدل معروف را بررسی می‌کنیم:

#### ۱. پروگرامر USBasp محبوب جهانی و ارزان

این مدل یک استاندارد جهانی برای AVR است. ارزان، کوچک و ساده است.

- مزیت: قیمت بسیار مناسب و یادگیری نحوه سیم‌کشی ISP.
- عیب: درایور آن در ویندوز ۱۰ و ۱۱ گاهی به سختی نصب می‌شود و سوکت برای قرار دادن مستقیم میکروکنترلر ندارد (باید سیم‌کشی کنید).
- نرم‌افزار: معمولاً با نرم‌افزار ProgISP کار می‌کند.

## ۲. پروگرامر هوشمند هاتل (Hatel) محبوب حرفه‌ای‌ها در ایران

این یک محصول ایرانی بسیار باکیفیت است که بسیاری از دردسرهای مدل‌های چینی را حل کرده است.

- **مزیت:** نصب درایور بسیار آسان، دارای سوکت ZIF (سوکت سبز رنگ که میکروکنترلر مستقیم داخلش قرار می‌گیرد) و قابلیت تولید کلاک نجات‌دهنده (برای وقتی که فیوزبیت اشتباه تنظیم شده).
- **نرم‌افزار:** نرم‌افزار اختصاصی **Hatel Programmer** با منوی فارسی.

## بخش دوم: روش‌های اتصال سخت‌افزاری

برای ریختن برنامه دو راه دارید:

روش الف) استفاده از سوکت ZIF مخصوص هاتل:

این روش راحت‌ترین کار است. اهرم سوکت سبز رنگ روی پروگرامر را بالا می‌کشید، میکروکنترلر را طبق راهنمای روی بدنه جا می‌زنید و اهرم را پایین می‌دهید. تمام!

روش ب) استفاده از کابل ISP مخصوص USBasp و هاتل)

اگر میکروکنترلر روی برد لحیم شده باشد یا نخواهید آن را جابجا کنید، از روش ISP استفاده می‌کنیم. در این روش ۶ سیم از پروگرامر به پایه‌های میکروکنترلر وصل می‌شود:

۱. **MOSI** پروگرامر --> پایه ۶ میکرو (MOSI)
۲. **MISO** پروگرامر --> پایه ۷ میکرو (MISO)
۳. **SCK** پروگرامر --> پایه ۸ میکرو (SCK)
۴. **RST** پروگرامر --> پایه ۹ میکرو (RESET)
۵. **VCC** پروگرامر --> پایه ۱۰ میکرو (VCC)
۶. **GND** پروگرامر --> پایه ۱۱ میکرو (GND)

## بخش سوم: فیوزبیت‌ها (Fuse Bits)؛ بایوس میکروکنترلر

فیوزبیت‌ها تنظیمات سخت‌افزاری هستند که تعیین می‌کنند قلب میکروکنترلر با چه سرعتی بتپد.

هشدار: تنظیم اشتباه فیوزبیت‌ها می‌تواند میکروکنترلر را از کار بیندازد Brick کند.

ما در کدها نوشتیم `#define F_CPU 1000000UL` پس سخت‌افزار هم باید روی ۱ مگاهرتز تنظیم باشد.

- **حالت کارخانه (پیش‌فرض):** وقتی میکرو نو می‌خرید، روی نوسان‌ساز داخلی ۱

مگاهرتز تنظیم است. پس اگر دست نزنید، همه چیز درست است.

- **تغییر سرعت:** اگر بخواهید سرعت را بالا ببرید (مثلاً ۸ مگاهرتز داخلی یا کریستال

خارجی)، باید این کدها را تغییر دهید.

جدول تنظیمات برای: ATmega32

در نرم‌افزارهای پروگرامر، دو بخش به نام Low Byte و High Byte دارید:

| منبع کلاک     | Low Byte | High Byte | توضیحات                           |
|---------------|----------|-----------|-----------------------------------|
| داخلی ۱MHz    | 0xE1     | 0xD9      | پیش‌فرض کارخانه (توصیه برای شروع) |
| داخلی ۸MHz    | 0xE4     | 0xD9      | سرعت بالاتر بدون قطعه اضافی       |
| کریستال خارجی | 0xFF     | 0xD9      | اگر کریستال کوارتز وصل کرده‌اید   |

## بخش چهارم: مراحل گام‌به‌گام اجرا

بیایید کدی که نوشته‌ایم را روی میکرو بریزیم:

۱. ساخت فایل Hex: در Atmel Studio دکمه F7 (Build) را بزنید. فایل hex در

پوشه پروژه ساخته می‌شود.

۲. اتصال: پروگرامر را به کامپیوتر و میکروکنترلر وصل کنید.

۳. اجرای نرم افزار:

○ اگر **USBasp** دارید: نرم افزار **ProgISP** را باز کنید. چیپ را **ATmega32** انتخاب کنید. دکمه **Load Flash** را زده و فایل **hex** را بیاورید. دکمه **Auto** را بزنید.

○ اگر **Hatel** دارید: نرم افزار **هاتل** را باز کنید. دکمه انتخاب قطعه را بزنید **(ATmega32)**. دکمه **Load** را زده و فایل را انتخاب کنید. دکمه **Program** را بزنید.

۴. پایان: اگر نوار سبز رنگ پر شد، برنامه با موفقیت ریخته شده است.

### بخش پنجم: فوت کوزه گری؛ نکات اجرایی مهم

وقتی وارد دنیای واقعی می شوید، با چالش هایی روبرو می شوید که در پروتئوس وجود نداشتند:

۱. ولتاژ تغذیه و افت ولتاژ: (Voltage Drop)

میکروکنترلر **ATmega32** استاندارد با ۵ ولت کار می کند.

- اگر ولتاژ منبع تغذیه شما (مثلاً باتری یا آداپتور) ضعیف شود و به زیر ۴ ولت یا ۲.۷ ولت برسد، میکروکنترلر ممکن است ریست شود یا رفتارهای عجیب (هنگ کردن) نشان دهد. (همینطور سعی کنید ولتاژ رو کمی کمتر از ۵ روی منبع تغذیه انتخاب کنید چون خطای منبع تغذیه و ضعیف بودن برخی از ورژن ها باعث زودتر سوختن ایسی شما میشه مثلاً ۴.۸ ولت خوبه)

- نکته: همیشه یک خازن حداقل ۱۰۰ نانو فاراد (100nF) دقیقاً کنار پایه های **VCC** و **GND** میکروکنترلر قرار دهید تا نویز ولتاژ را بگیرد. (خیلی وقتا میان ۱۰۰ یا ۱۰۰۰ میکروفاراد توی مدار های بزرگتر میزارن که نویز رو کاملاً حذف کنه)

۲. پایه ریست: (Reset Pin)

پایه شماره ۹ (Reset) بسیار حساس است. اگر دستتان یا یک نويز محیطی به آن برخورد، میکرو ریست می‌شود.

- راه حل: همیشه این پایه را با یک مقاومت ۱۰ کیلو اهم به برق مثبت (VCC) وصل کنید (Pull-up). (این کار پایداری مدار را تضمین می‌کند.

۳. داغ شدن میکروکنترلر:

- اگر به محض روشن کردن مدار، میکروکنترلر داغ شد، سریعاً برق را قطع کنید!
- احتمالاً جای VCC و GND را برعکس وصل کرده‌اید یا پایه‌های خروجی اتصال کوتاه شده‌اند.

**تبریک می‌گوییم!** شما اکنون یک مهندس سیستم‌های نهفته هستید. از مفاهیم تئوری شروع کردید، کد زدید، شبیه‌سازی کردید و حالا محصول نهایی خود را ساختید.